

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Implementierung und Matlab-Praxis 4:

Grafische Darstellung

Inhalt

4. Grafische Darstellung

- 4.1 Einführung
- 4.2 Formatierung
- 4.3 Speichern/Export
- 4.4 2-dim. Graphen
- 4.5 Diagramme
- 4.6 3-dim. Grafiken
- 4.7 Schnittgrafiken
- 4.8 Spezielle Grafiken
- 4.9 Darstellen von Funktionen
- 4.10 Übersicht
- 4.11 GUIs

4. Grafische Darstellung – 4.1 Einführung

Grafik unter Matlab:

- Matlab hat sehr mächtige Befehle zur Darstellung von Grafiken.
- Zur Erstellung „schöner“ Grafiken ist eine gute Kenntnis der Grafikbefehle und insbesondere der Formatierungsbefehle notwendig.
- Wir haben bereits den `plot`-Befehl kennengelehrt.

```
>> x = linspace(0, 2*pi, 100);  
>> y = sin(x) + sin(2*x);  
>> plot(x,y);
```

```
>> max(y)
```

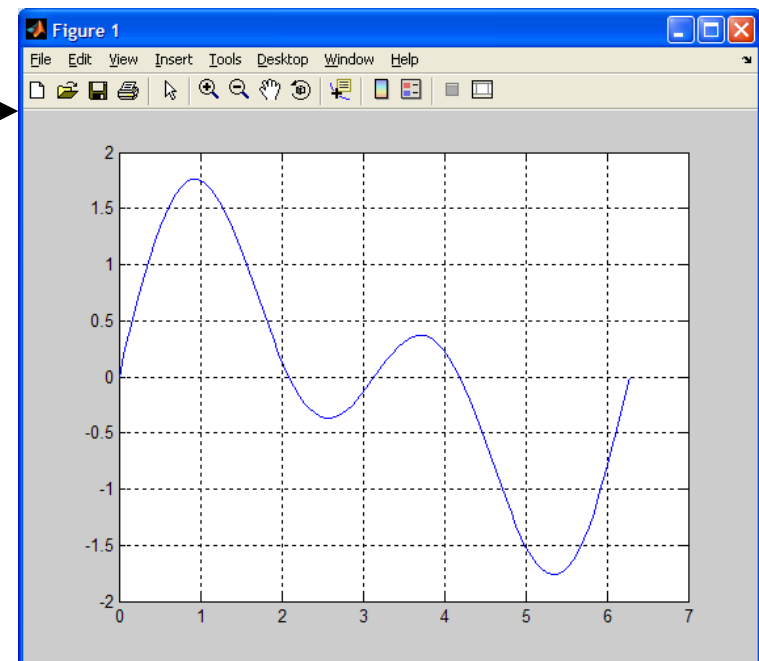
```
ans =
```

```
1.7596
```

```
>> mean(y)
```

```
ans =
```

```
-9.0615e-017
```



4. Grafische Darstellung – 4.2 Formatierung

`plot` kann auch mehrere Polygone mit einer Anweisung zeichnen, wenn diese durch eine Matrix spezifiziert werden:

```
>> x=[1,2,3;4,5,6;7,8,9]
```

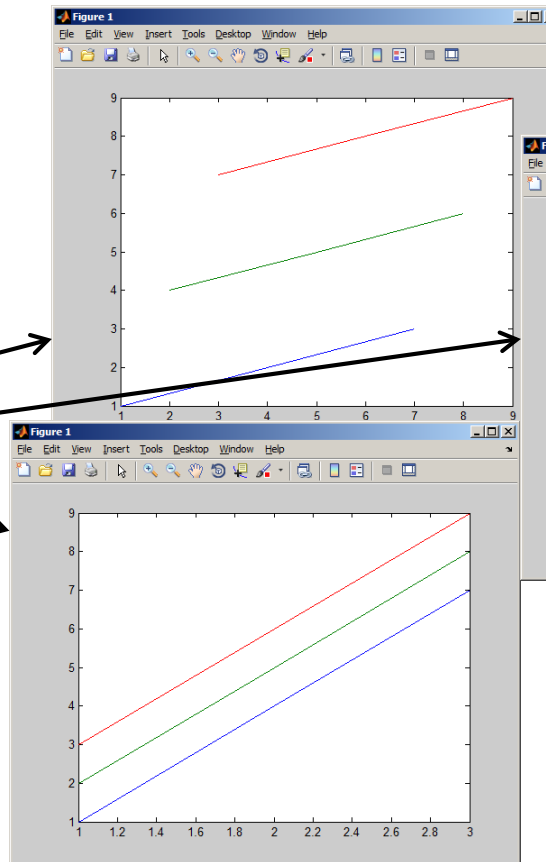
```
x =
```

1	2	3
4	5	6
7	8	9

```
>> plot(x,x')
```

```
>> plot(x',x)
```

```
>> plot(x(1,:),x)
```



4. Grafische Darstellung – 4.2 Formatierung

Elementare Anweisung für Grafiken:

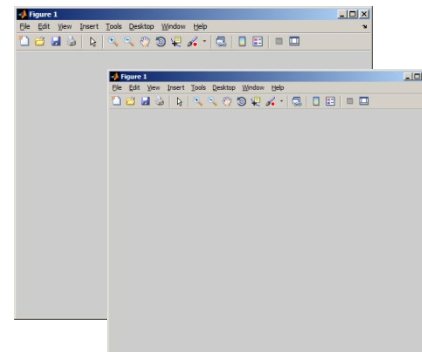
```
>>figure(<fighandle>);
```

→ öffnen eine Grafikfensters mit dem „Handle“ (der Nummer) `<fighandle>`. Sollte bereits ein Grafikfenster mit dem Handle `<fighandle>` existieren, wird dies in den Vordergrund gebracht zum „aktuellen“ Grafikfenster gemacht.

Wichtig: Alle Grafikbefehle beziehen sich immer auf das aktuelle Grafikfenster. Die Anweisung `figure(<fighandle>)` dient daher auch zu umschalten zwischen den Grafikfenstern bei der Bearbeitung.

Beispiel:

```
figure(1);  
...  
figure(2);
```



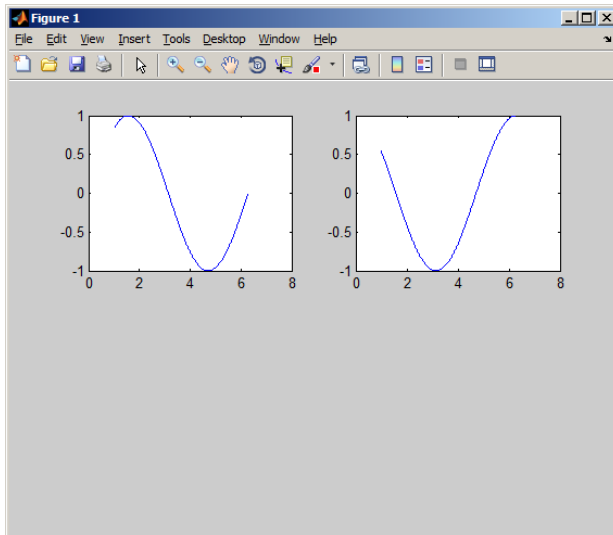
4. Grafische Darstellung – 4.2 Formatierung

```
>> subplot(<m>,<n>,<numb>)
```

→ Der `subplot` Befehl teilt ein Grafikfenster auf, so dass `<m> x <n>` Einzelgrafiken (genannt „axis“) darstellbar sind. Mit `<numb>` wird die Grafik („axis“) ausgewählt (zeilenweise Indizierung).

Beispiel:

```
>> subplot(2,2,1), plot(1:0.01:2*pi), sin(1:0.01:2*pi))  
>> subplot(2,2,2), plot(1:0.01:2*pi), cos(1:0.01:2*pi))
```



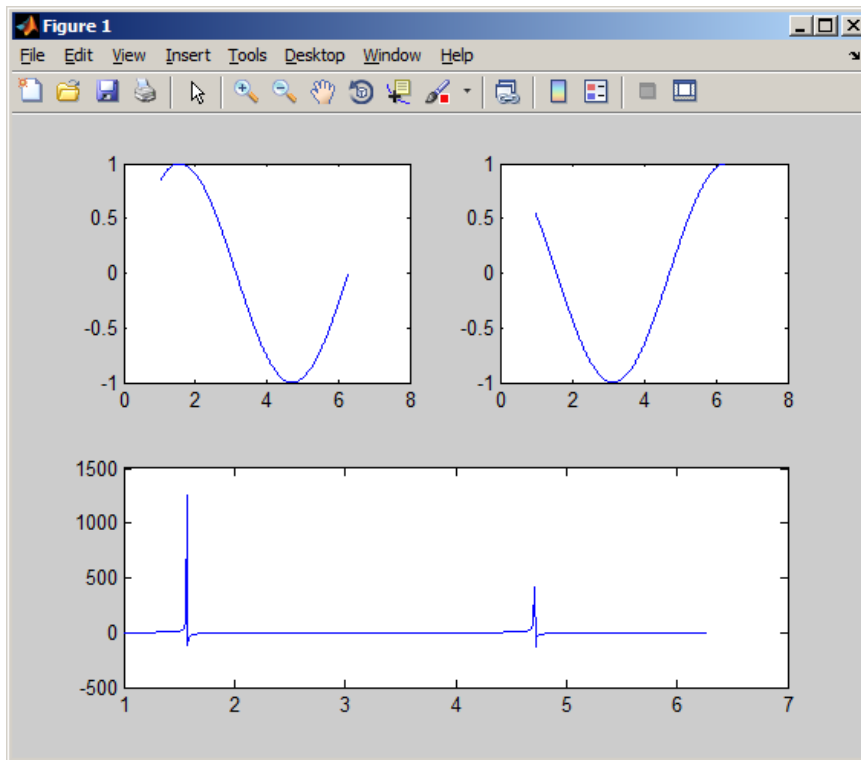
Beachte:

`hold on` ist bei unterschiedliche Subplots nicht notwendig

4. Grafische Darstellung – 4.2 Formatierung

Es funktioniert auch:

```
>> subplot(2,2,1), plot((1:0.01:2*pi),sin(1:0.01:2*pi))  
>> subplot(2,2,2), plot((1:0.01:2*pi),cos(1:0.01:2*pi))  
>> subplot(2,1,2), plot((1:0.01:2*pi),tan(1:0.01:2*pi))
```



4. Grafische Darstellung – 4.2 Formatierung

Hilfreiche Befehle für „Grafikhandles“:

- `>> ghandle = gcf` → Liefert das Handle des aktuellen Grafikfenster
- `>> ahandle= gca` → Liefert das Handle der aktuellen Grafik (Axis)
- `>> ohandle= gco` → Liefert das Handle des aktuellen Objekts (object)
- `>> bohandle= gcbo` → Liefert das Handle des aktuellen Grafikobjekts
- `>> ohandle= findobj('...', '...', '...');`
→ Sucht ein Objekt mit den Attributeigenschaften
- `>> clf(<ahandle>)` → Löscht ein Grafikfenster (Figure)
- `>> cla(<ahandle>)` → Löscht eine Grafik (Axis)
- `>> clf(gcf)` → Löscht das aktuelle Grafikfenster bzw. Grafik



4. Grafische Darstellung – 4.2 Formatierung

```
>> handles = allchild(<handle>)
```

→ Liefert die Handles der Unterobjekte zu dem Objekt <handle>

```
>> handles = findall(<handlelist>)
```

→ Liefert die Handles der Unterobjekte zu allen Objekten der
<handlelist>

Hinweis:

Will man (Grafik-)Objekte später gezielten mit dem `set`-Befehl manipulieren, empfiehlt es sich die Handle direkt beim Anlegender der Objekte zu speichern.

4. Grafische Darstellung – 4.2 Formatierung

Elementare Befehle zum Formatieren von Grafiken:

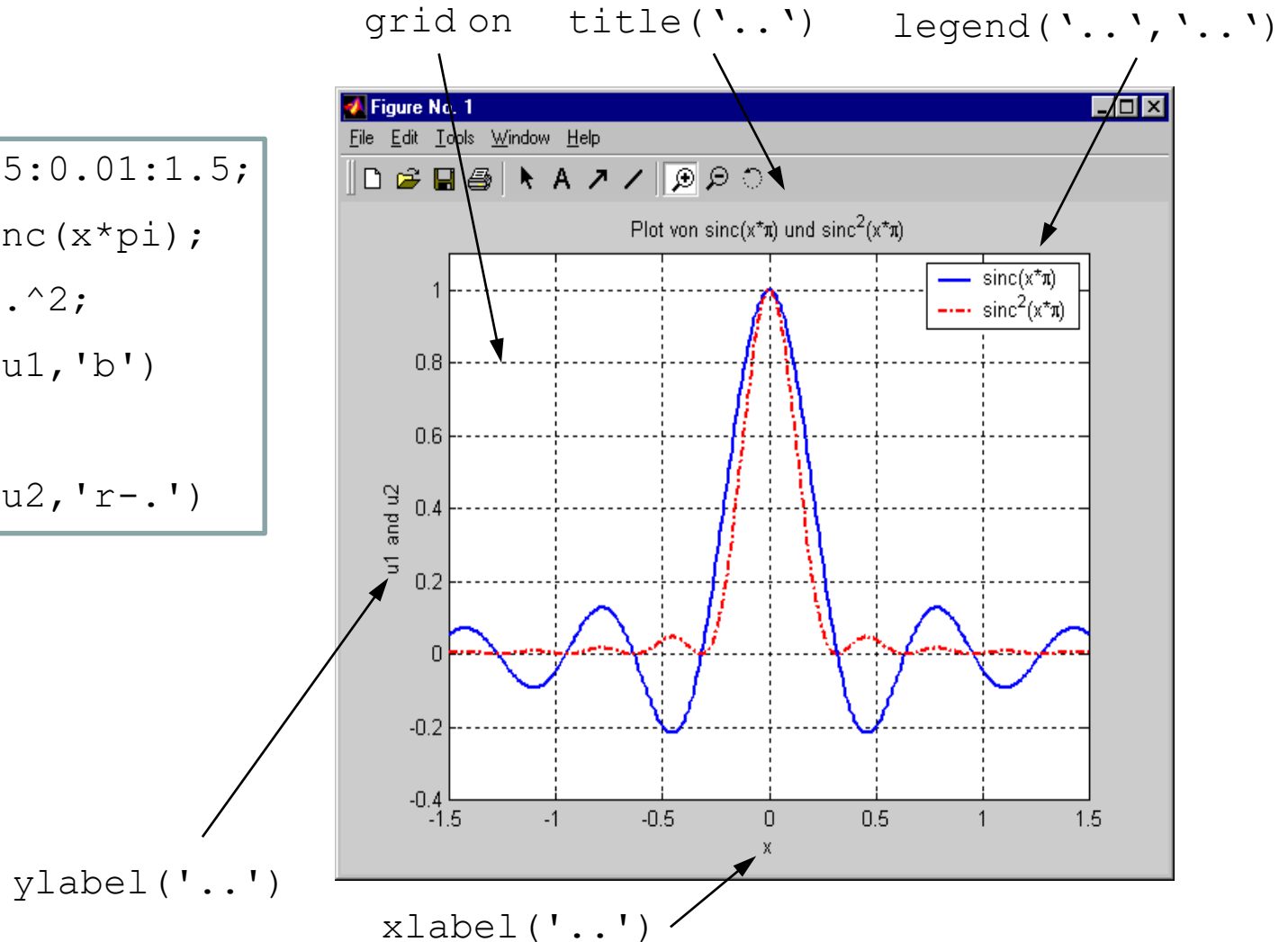
<code>title(<string>)</code>	→ Überschrift setzen
<code>xlabel(<string>)</code>	→ Beschriftung der x-Achse
<code>ylabel(<string>)</code>	→ Beschriftung der x-Achse
<code>text (<xposition>,<yposition>,<string>)</code>	
	→ Freier Text
<code>gtext</code>	→ Freier Text mit Cursor setzen
<code>legend (<string1>, <string2>, <string3>...)</code>	
	→ Darstellung der Legende
<code>grid on (off)</code>	→ Ein(Aus-)schalten des Gitters
<code>hold on (off)</code>	→ Halten bereits gezeichneter Elemente
<code>axis([xmin xmax ymin ymax])</code>	
	→ Festlegen der Darstellungsgrenzen
<code>annotation(<...>)</code>	→ Hinzufügen von Objekten (z. B. Pfeilen)



4. Grafische Darstellung – 4.2 Formatierung

Beispiel:

```
x = -1.5:0.01:1.5;  
u1 = sinc(x*pi);  
u2 = u1.^2;  
plot(x,u1,'b')  
hold on  
plot(x,u2,'r-.')
```



4. Grafische Darstellung – 4.2 Formatierung

Die Formatierungsanweisungen haben teilweise noch erheblich mehr Optionen:

Beispiele:

`>> axis square` → quadratische Darstellung

`>> axis normal` → Zurück zum Normalmodus

`>> axis tight` → Achsen durch Daten festlegen

`>> legend hide` → Legende ausblenden

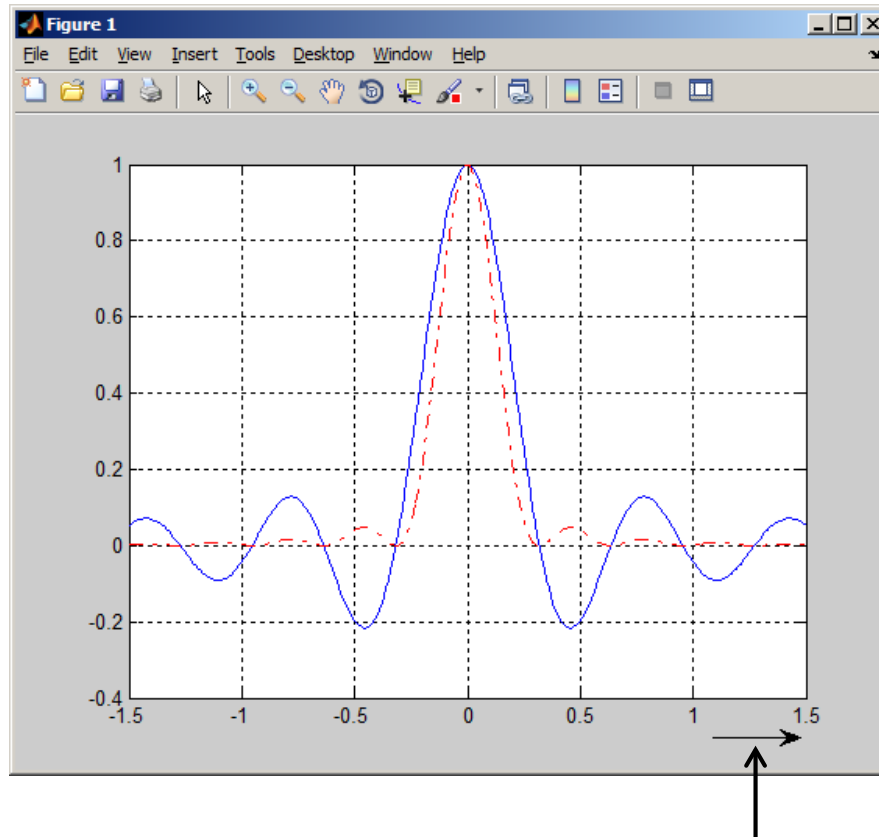
`>> legend(..., 'Location', <location>)` → Position festlegen

(<location> ... 'North', 'South', 'East', 'West', 'NorthEast')

Für weitere Optionen bitte die Hilfe bemühen!!



4. Grafische Darstellung – 4.2 Formatierung



Hinweis:

Die Positionsangaben beziehen sich auf das Grafikfenster und nicht auf die Grafik.

```
>> annotation(gcf,'arrow',[0.8,0.9],[0.05,0.05]);
```

4. Grafische Darstellung – 4.2 Formatierung

Wichtig: Die Formatierungsbefehle können auch mit Angabe des Grafikhandles ausgeführt werden:

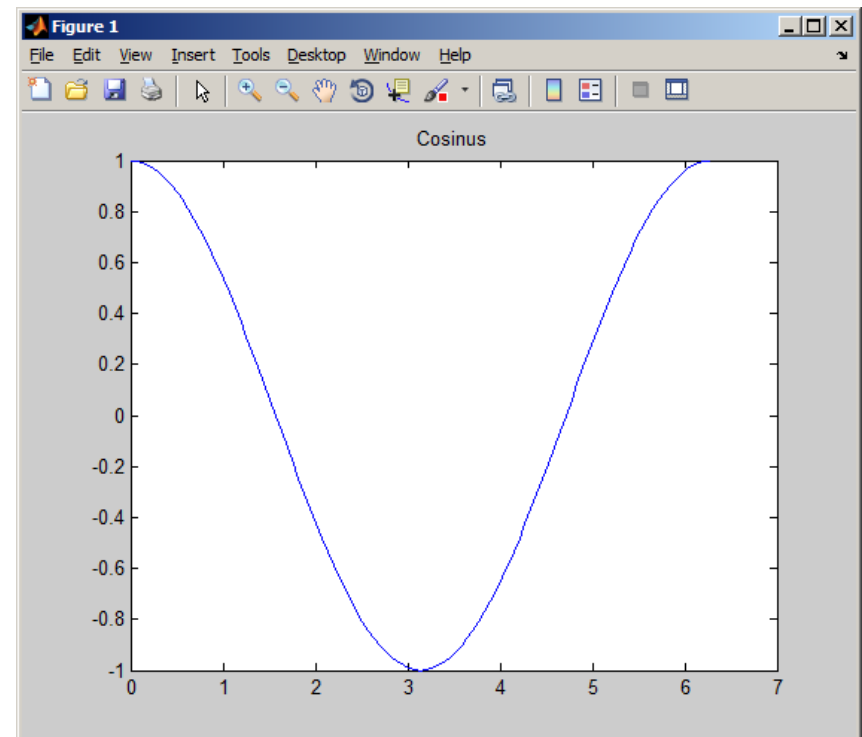
```
>> plot((1:2*pi/10:2*pi),cos(1:2*pi/10:2*pi))
```

```
>> ahandle = gca
```

```
ahandle =
```

```
170.0170
```

```
>> title(ahandle, 'Cosinus');
```



4. Grafische Darstellung – 4.2 Formatierung

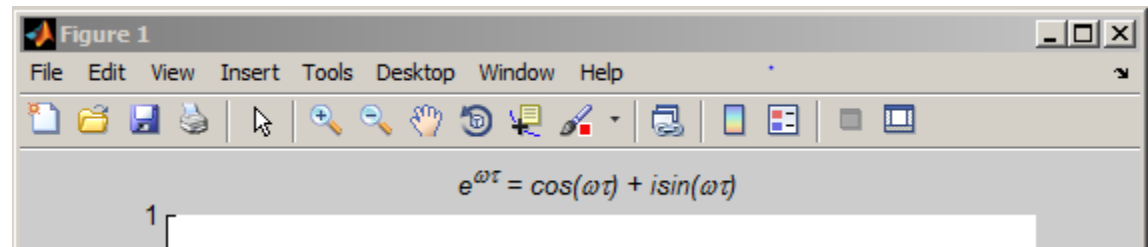
Strings für Formatierungsbefehle ermöglichen auch das Anzeigen...

- von griechischen Buchstaben
- hoch- oder tiefgestelltem Text
- unterschiedlichen Formatierungen

mit Hilfe der TeX-Syntax (siehe z. B. Wikipedia)

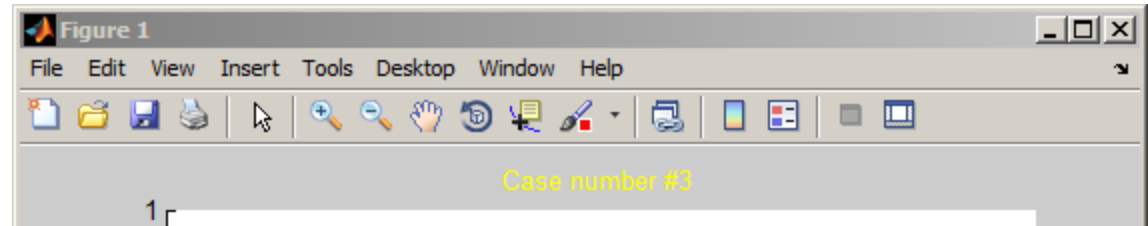
Beispiele:

```
>> title('\ite^{\omega\tau} = cos(\omega\tau) +  
isin(\omega\tau)')
```

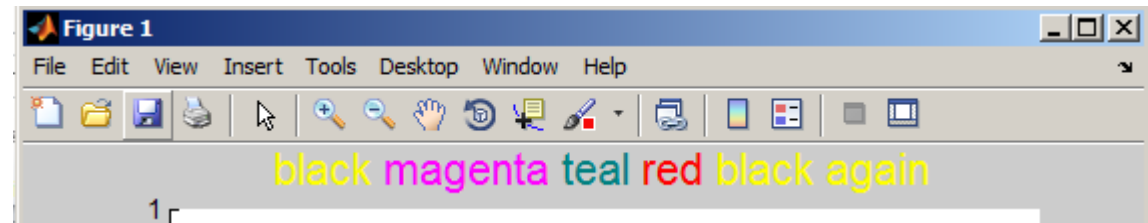


4. Grafische Darstellung – 4.2 Formatierung

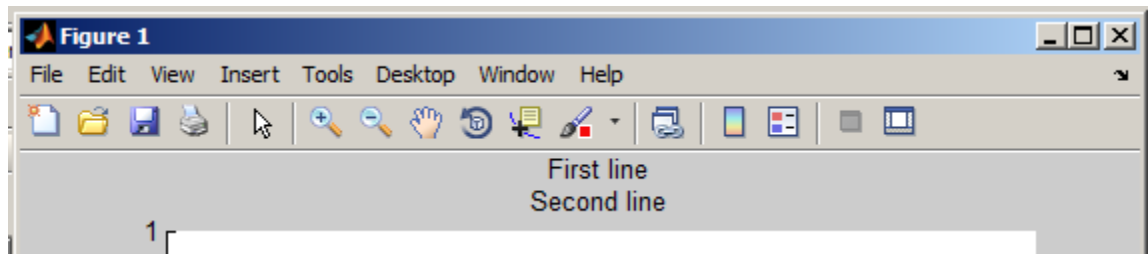
```
>> title(['Case number #',int2str(n)],'Color','y')
```



```
>> title(['\fontsize{16}black {\color{magenta}magenta '...  
'\color[rgb]{0 .5 .5}teal \color{red}red} black again'])
```



```
>>title({'First line';'Second line'})
```



4. Grafische Darstellung – 4.2 Formatierung

... aus der Matlab-Hilfe

Character Sequence	Symbol	Character Sequence	Symbol	Character Sequence	Symbol
<code>\alpha</code>	α	<code>\upsilon</code>	υ	<code>\sim</code>	$\sim$
<code>\beta</code>	β	<code>\phi</code>	Φ	<code>\leq</code>	$\leq$
<code>\gamma</code>	γ	<code>\chi</code>	χ	<code>\infty</code>	∞
<code>\delta</code>	δ	<code>\psi</code>	ψ	<code>\clubsuit</code>	$\clubsuit$
<code>\epsilon</code>	ϵ	<code>\omega</code>	ω	<code>\diamondsuit</code>	$\diamondsuit$
<code>\zeta</code>	ζ	<code>\Gamma</code>	Γ	<code>\heartsuit</code>	$\heartsuit$
<code>\eta</code>	η	<code>\Delta</code>	Δ	<code>\spadesuit</code>	$\spadesuit$
<code>\theta</code>	θ	<code>\Theta</code>	Θ	<code>\leftrightarrow</code>	$\leftrightarrow$
<code>\vartheta</code>	ϑ	<code>\Lambda</code>	Λ	<code>\leftarrow</code>	$\leftarrow$
<code>\iota</code>	ι	<code>\Xi</code>	Ξ	<code>\uparrow</code>	$\uparrow$
<code>\kappa</code>	κ	<code>\Pi</code>	Π	<code>\rightarrow</code>	$\rightarrow$
<code>\lambda</code>	λ	<code>\Sigma</code>	Σ	<code>\downarrow</code>	$\downarrow$
<code>\mu</code>	μ	<code>\Upsilon</code>	Υ	<code>\circ</code>	$\circ$
<code>\nu</code>	ν	<code>\Phi</code>	Φ	<code>\pm</code>	$\pm$
<code>\xi</code>	ξ	<code>\Psi</code>	Ψ	<code>\geq</code>	$\geq$
<code>\pi</code>	π	<code>\Omega</code>	Ω	<code>\propto</code>	$\propto$
<code>\rho</code>	ρ	<code>\forall</code>	$\forall$	<code>\partial</code>	∂
<code>\sigma</code>	σ	<code>\exists</code>	$\exists$	<code>\bullet</code>	$\bullet$
<code>\varsigma</code>	ς	<code>\div</code>	$\div$	<code>\div</code>	$\div$

4. Grafische Darstellung – 4.2 Formatierung

Weitere Attribute für Texte:

`\bf`

→ Bold

`\it`

→ Italic

`\rm`

→ Normal

`\fontname{fontname}`

→ Schriftart

`\fontsize{fontsize}`

→ Schriftgröße

`\color{colorSpec}`

→ Schriftfarbe



4. Grafische Darstellung – 4.2 Formatierung

LaTeX:

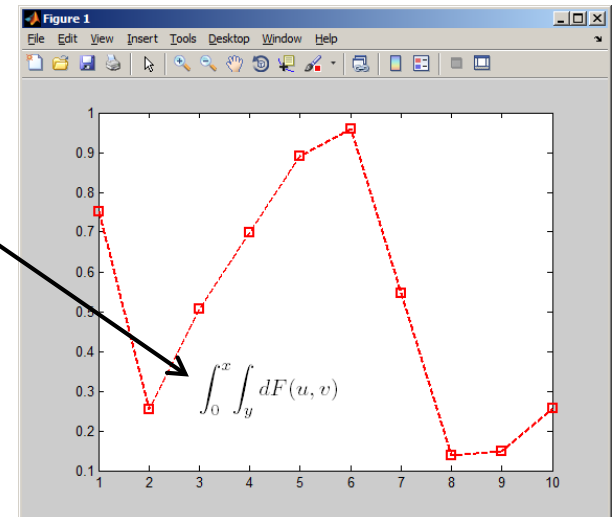
Der Textinterpreter kann auch auf LaTeX umgeschaltet werden:

```
>> h = text('Interpreter','latex',...  
            'String','$$\int_0^x \int_y dF(u,v)$$',...  
            'Position',[.5 .5],...  
            'FontSize',16)
```

Hinweis:

Die Eingabe erfolgte hier ausschließlich durch die Schlüsselwortnotation:

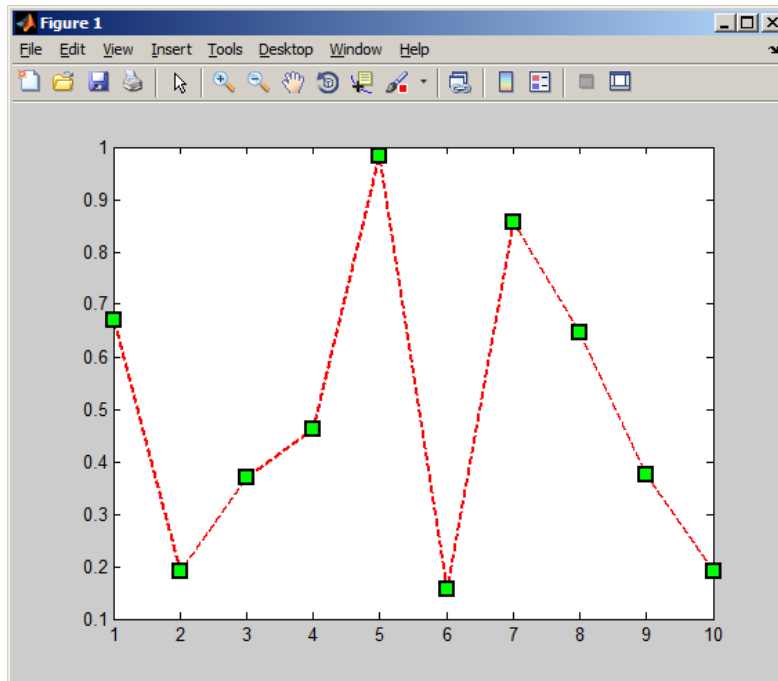
```
(... 'String','$$\int_0^x \int_y dF(u,v)$$',...)
```



4. Grafische Darstellung – 4.2 Formatierung

Es können mit dem plot-Befehl auch noch Attribute übergeben werden:

```
>> phandle = plot(linspace(1,10,10),rand(10,1),...  
                  '--rs','LineWidth',2,...  
                  'MarkerEdgeColor','k',...  
                  'MarkerFaceColor','g',...  
                  'MarkerSize',10)
```



phandle ist da „Handle“ des Plots

4. Grafische Darstellung – 4.2 Formatierung

```
>> get(phandle)
      DisplayName: ''
      Annotation: [1x1
hg.Annotation]
      Color: [1 0 0]
      LineStyle: '--'
      LineWidth: 2
      Marker: 'square'
      MarkerSize: 10
      MarkerEdgeColor: [0 0 0]
      MarkerFaceColor: [0 1 0]
      XData: [1 2 3 4 5 6
              7 8 9 10]
      YData: [1x10 double]
      ZData: [1x0 double]
      BeingDeleted: 'off'
      ButtonDownFcn: []
      Children: [0x1 double]
      Clipping: 'on'
```

```
CreateFcn: []
      DeleteFcn: []
      BusyAction: 'queue'
      HandleVisibility: 'on'
      HitTest: 'on'
      Interruptible: 'on'
      Selected: 'off'
      SelectionHighlight: 'on'
      Tag: ''
      Type: 'line'
      UIContextMenu: []
      UserData: []
      Visible: 'on'
      Parent: 170.0375
      XDataMode: 'manual'
      XDataSource: ''
      YDataSource: ''
      ZDataSource: ''
```

4. Grafische Darstellung – 4.2 Formatierung

Die `get`- und `set`-Anweisung:

Die `get`-Anweisung liefert spezifische Eigenschaften eines Grafikfensters oder einer Grafik. Beispiel:

```
>> get(gca, 'Box')
```

```
ans =
```

```
on      → Rahmen „eingeschaltet“
```

Alle Attribute werden angezeigt durch:

```
>> get(gca)
ActivePositionProperty = outerposition
ALim = [0 1]
ALimMode = auto
AmbientLightColor = [1 1 1]
Box = on
CameraPosition = [3.5 0 17.3205]
CameraPositionMode = auto
CameraTarget = [3.5 0 0]
```

4. Grafische Darstellung – 4.2 Formatierung

```
CameraTargetMode = auto
CameraUpVector = [0 1 0]
CameraUpVectorMode = auto
CameraViewAngle = [6.60861]
CameraViewAngleMode = auto
CLim = [0 1]
CLimMode = auto
Color = [1 1 1]
CurrentPoint = [ (2 by 3) double array]
ColorOrder = [ (7 by 3) double array]
DataAspectRatio = [3.5 1 1]
DataAspectRatioMode = auto
DrawMode = normal
FontAngle = normal
FontName = Helvetica
FontSize = [10]
FontUnits = points
FontWeight = normal
GridLineStyle = :
Layer = bottom
LineStyleOrder = -
LineWidth = [0.5]
MinorGridLineStyle = :
NextPlot = replace
```

4. Grafische Darstellung – 4.2 Formatierung

```
OuterPosition = [0 0 1 1]
PlotBoxAspectRatio = [1 1 1]
PlotBoxAspectRatioMode = auto
Projection = orthographic
Position = [0.13 0.11 0.775 0.815]
TickLength = [0.01 0.025]
TickDir = in
TickDirMode = auto
TightInset = [0.0464286 0.0404762 0.00892857 0.0547619]
Title = [172.017]
Units = normalized
View = [0 90]
XColor = [0 0 0]
XDir = normal
XGrid = off
XLabel = [173.017]
XAxisLocation = bottom
XLim = [0 7]
XLimMode = auto
XMinorGrid = off
XMinorTick = off
XScale = linear
XTick = [ (1 by 8) double array]
XTickLabel =
```

4. Grafische Darstellung – 4.2 Formatierung

```
0
1
2
3
4
5
6
7
XTickLabelMode = auto
XTickMode = auto
YColor = [0 0 0]
YDir = normal
YGrid = off
YLabel = [174.017]
YAxisLocation = left
YLim = [-1 1]
YLimMode = auto
YMinorGrid = off
YMinorTick = off
YScale = linear
YTick = [ (1 by 11) double array]
YTickLabel = [ (11 by 4) char array]
YTickLabelMode = auto
YTickMode = auto
```

4. Grafische Darstellung – 4.2 Formatierung

```
ZColor = [0 0 0]
ZDir = normal
ZGrid = off
ZLabel = [175.017]
ZLim = [-1 1]
ZLimMode = auto
ZMinorGrid = off
ZMinorTick = off
ZScale = linear
ZTick = [-1 0 1]
ZTickLabel =
ZTickLabelMode = auto
ZTickMode = auto
```

```
BeingDeleted = off
ButtonDownFcn =
Children = [171.019]
Clipping = on
CreateFcn =
DeleteFcn =
BusyAction = queue
HandleVisibility = on
HitTest = on
Interruptible = on
```

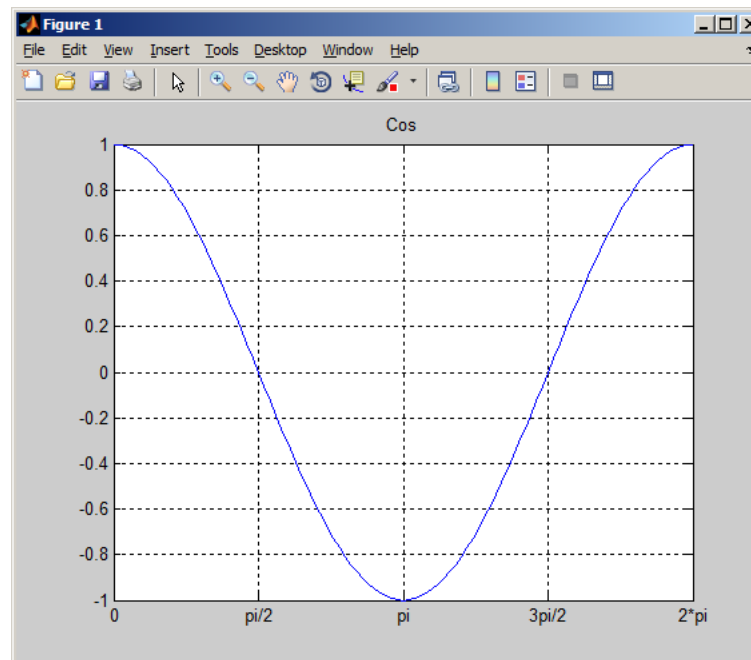
4. Grafische Darstellung – 4.2 Formatierung

```
Parent = [1]
Selected = off
SelectionHighlight = on
Tag =
Type = axes
UIContextMenu = []
UserData = []
Visible = on
```

4. Grafische Darstellung – 4.2 Formatierung

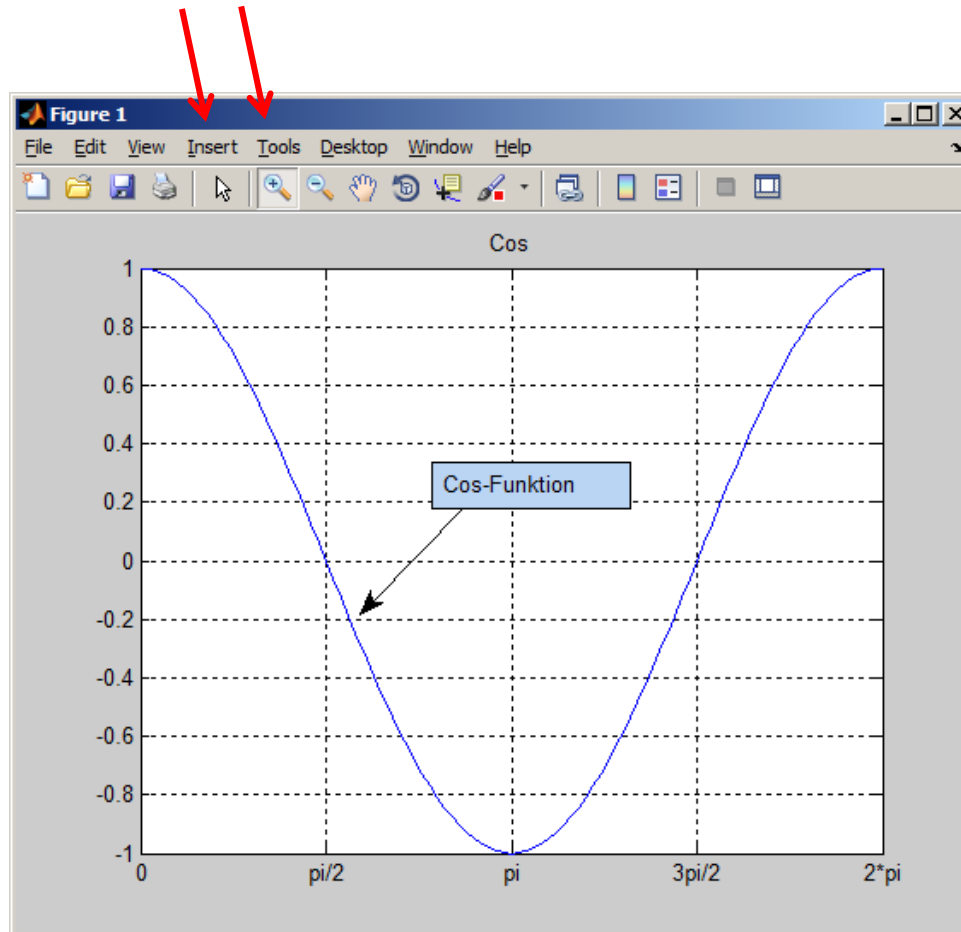
Die `set`-Anweisung setzt spezifische Eigenschaften eines Grafikfensters oder einer Grafik. Beispiel:

```
>> set(gca,'XTickLabel',{'0', 'pi', '2*pi'})
>> grid
>> plot((0:2*pi/100:2*pi),cos(0:2*pi/100:2*pi))
>> set(gca,'XTick',[0,0.5*pi,pi,1.5*pi,2*pi])
>> set(gca,'XTickLabel',{'0','pi/2','pi','3pi/2','2*pi'})
>> axis([0,2*pi,-1,1])
>> grid on
>> title('Cos')
```



4. Grafische Darstellung – 4.2 Formatierung

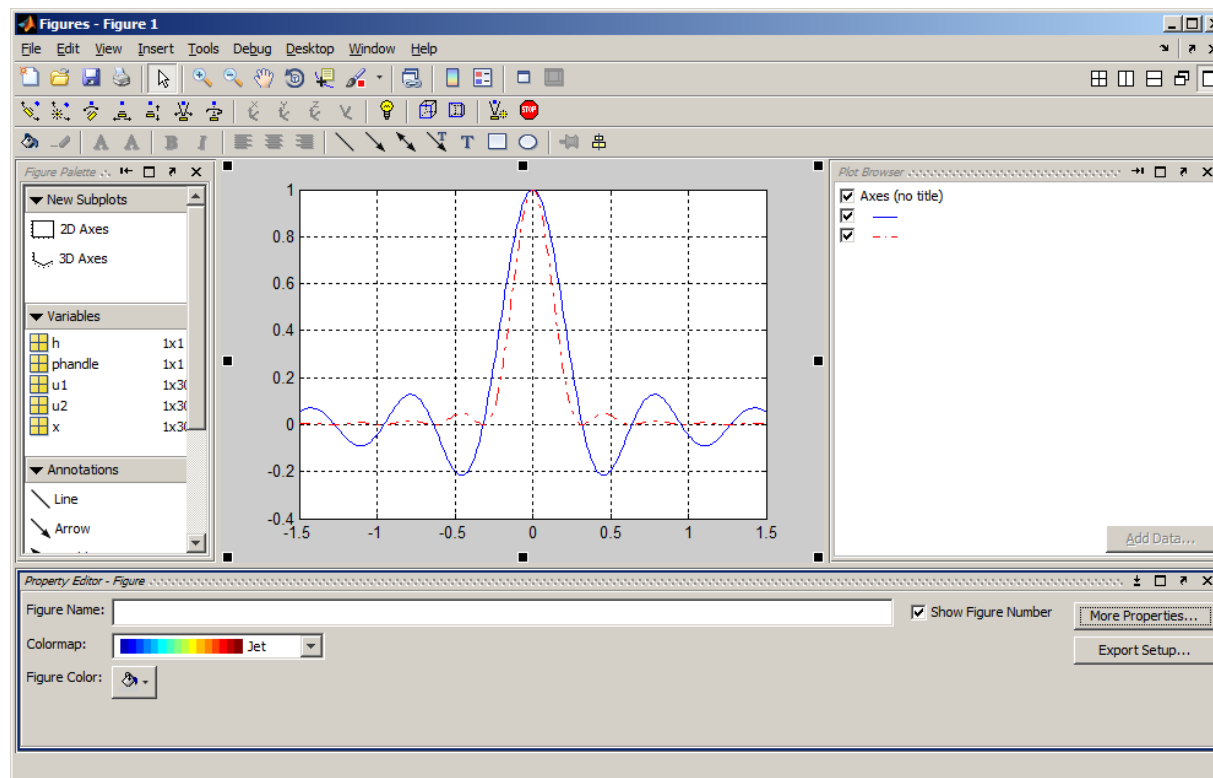
Die Grafikfenster neuerer Matlab-Versionen bieten auch interaktiven Möglichkeiten zu Editierung:



4. Grafische Darstellung – 4.2 Formatierung

Das Plot-Tool:

Der Befehl `plottools` (oder über das Menü der Grafikfenster) öffnet für ein bestehendes Fenster das Interaktive Plot-Tool von Matlab, mit dem Grafiken „manuell“ nachbearbeitet werden können.



4. Grafische Darstellung – 4.3 Speichern/Export

Speichern und Export von Grafiken:

```
saveas (<ghandle>,<filename>,<format>);
```

Speichert die Grafik <ghandle> unter dem Namen <filename> im Format <format>.

Formate (<format>):

'fig'	→	Binärfile (.fig).Kann mit open geladen werden
'm'	→	Binärfile und ausführbares m-File zu laden
'mmat'	→	Ausführbares m-File (größer datenmengen als .mat)

Der print-Befehl (drucken auch in Datei) bietet noch mehr Optionen für den Export. Siehe Hilfe.



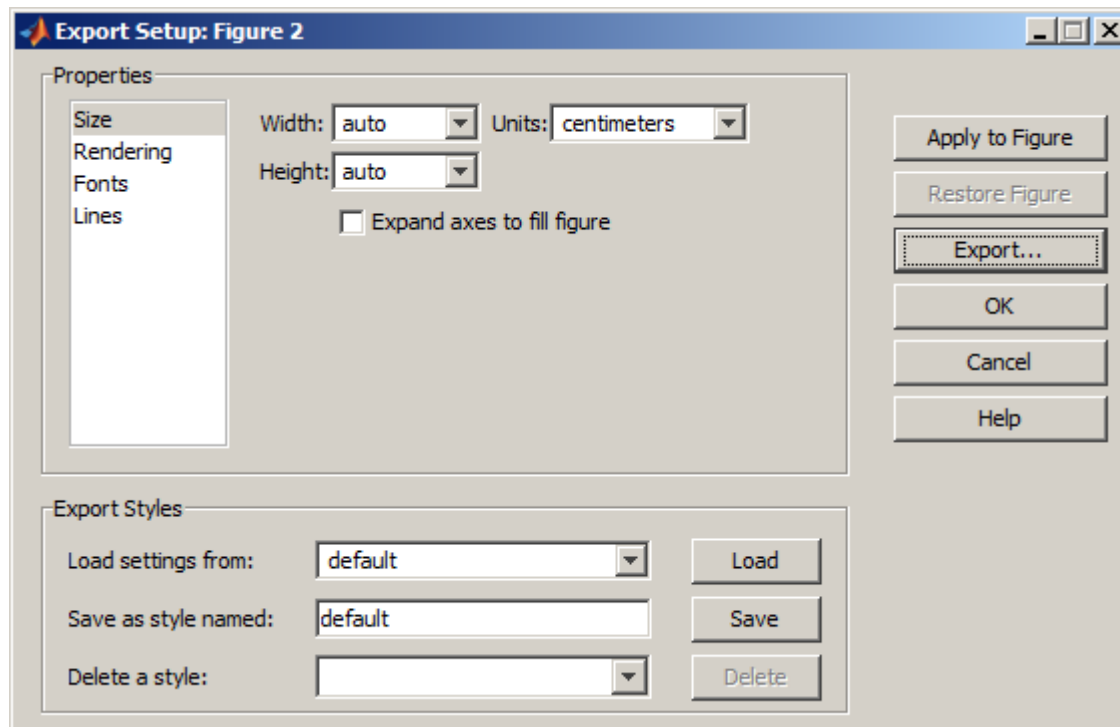
4. Grafische Darstellung – 4.3 Speichern/Export

Inhalt des m-File:

```
function h1 = Grafic2()  
% This is the machine-generated representation of a Handle Graphics object  
...  
% are incompatible with the M-file/MAT-file format, and should be saved as  
% FIG-files.  
  
load Grafic2.mat  
appdata = [];  
appdata.scribegui_snaptogrid = mat{1};  
appdata.uitools_FigureToolManager = mat{2};  
appdata.FileMenuFcnLastExportedAsType = mat{3};  
appdata.SavedVisible = mat{4};  
  
h1 = figure(...  
    'PaperUnits',get(0,'defaultfigurePaperUnits'),...  
    'Color',get(0,'defaultfigureColor'),...  
    'Colormap',get(0,'defaultfigureColormap'),...  
    'FileName','C:\Users\Schulte\HS_OWL\Lehre\MatlabSeminar\Matlab\Kap4\Grafic.fig',...  
    'InvertHardcopy',get(0,'defaultfigureInvertHardcopy'),...  
    'PaperPosition',[0.6345175 6.345175 20.30456 15.22842],...  
    'PaperSize',get(0,'defaultfigurePaperSize'),...  
    'PaperType',get(0,'defaultfigurePaperType'),...  
    'Position',[807 280 560 420],...  
    'CreateFcn',{@local_CreateFcn, blanks(0), appdata} );
```

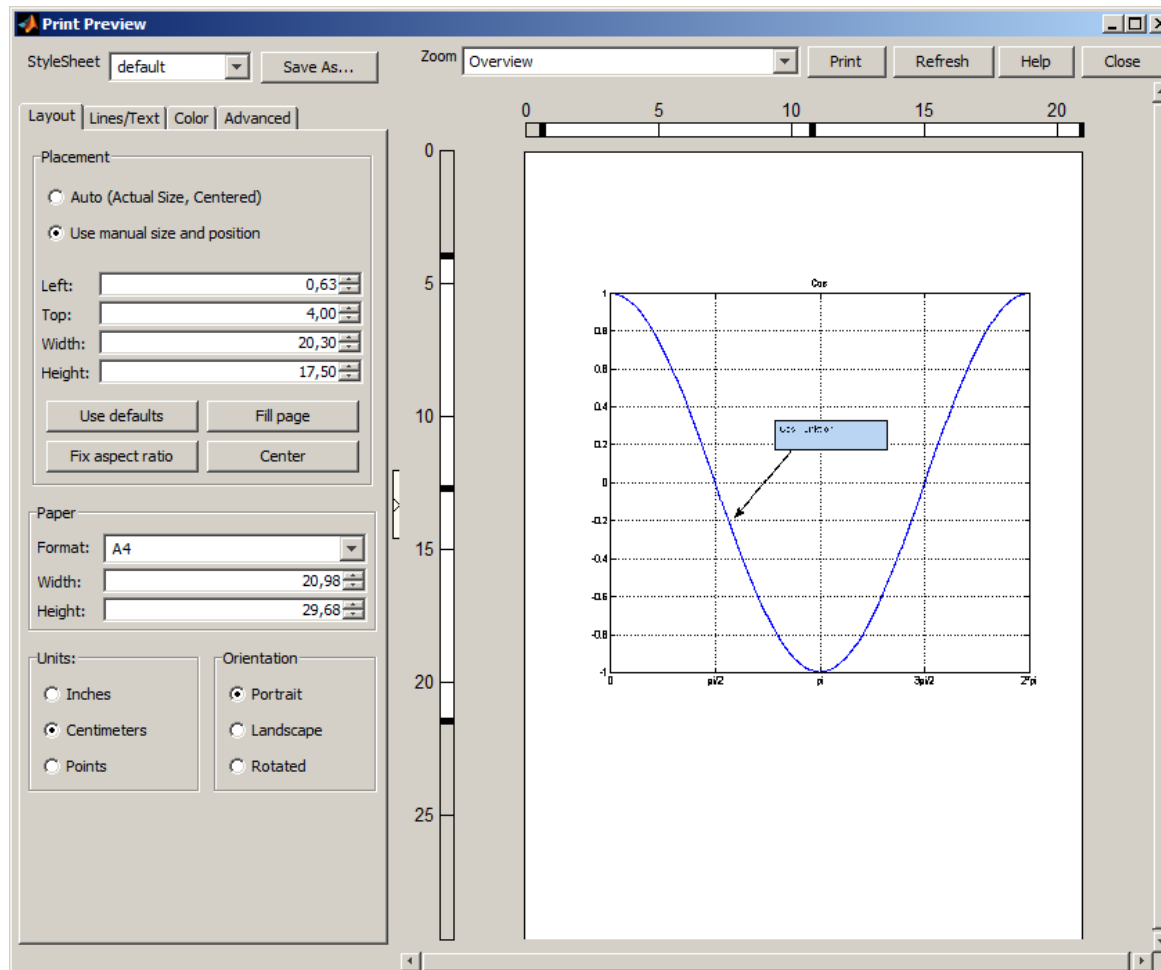
4. Grafische Darstellung – 4.3 Speichern/Export

...alternative kann der Export über das Menü (Export-Setup) durchgeführt werden:



4. Grafische Darstellung – 4.3 Speichern/Export

...genauso wie der Druck:



4. Grafische Darstellung – 4.3 Speichern/Export

Befehle zum Drucken:

<code>print</code>	→ Drucken und Speichern von Grafiken
<code>orient</code>	→ Papierorientierung
<code>printdlg</code>	→ Dialogfenster für das Drucken
<code>pagesetupdlg</code>	→ Dialogfenster für die Seiteneinstellungen
<code>printpreview</code>	→ Druckvorschau



4. Grafische Darstellung – 4.4 2-dim Graphen

Grafik-Funktionen für ebene (2-dim) Grafiken:

<code>plot</code>	→ Polygonzug
<code>semilogx</code>	→ Polygonzug (logarithm. Skalierung in x-Richtung)
<code>semilogy</code>	→ Polygonzug (logarithm. Skalierung in y-Richtung)
<code>loglog</code>	→ Polygonzug mit logarithm. Skalierung in x- und y-Richtung)
<code>polar</code>	→ Polygonzug in Polarkoordinaten
<code>plotyy</code>	→ Polygonzug mit unterschiedlicher Skalierung
<code>line</code>	→ Polygonzug mit beliebiger Skalierung (s. u.)
<code>stairs</code>	→ Stufendarstellung
<code>patch</code>	→ gefüllte Fläche
<code>fill</code>	→ gefüllte Fläche



4. Grafische Darstellung – 4.4 2-dim Graphen

Logarithmische Grafiken:

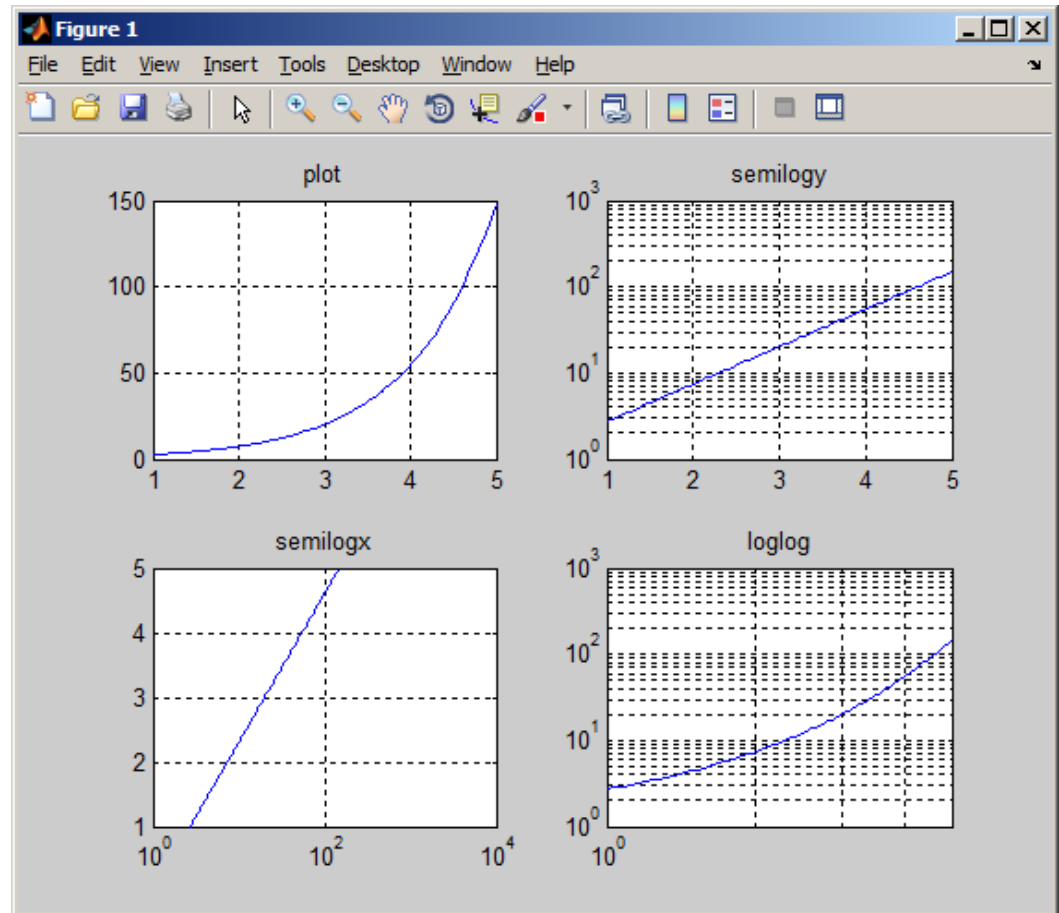
```
x=linspace(1,5,100);  
y=exp(x);
```

```
figure(1)  
subplot(2,2,1)  
plot(x,y);  
grid on;  
title('plot');
```

```
subplot(2,2,2)  
semilogy(x,y);  
grid on;  
title('semilogy');
```

```
subplot(2,2,3)  
semilogx(y,x);  
grid on;  
title('semilogx');
```

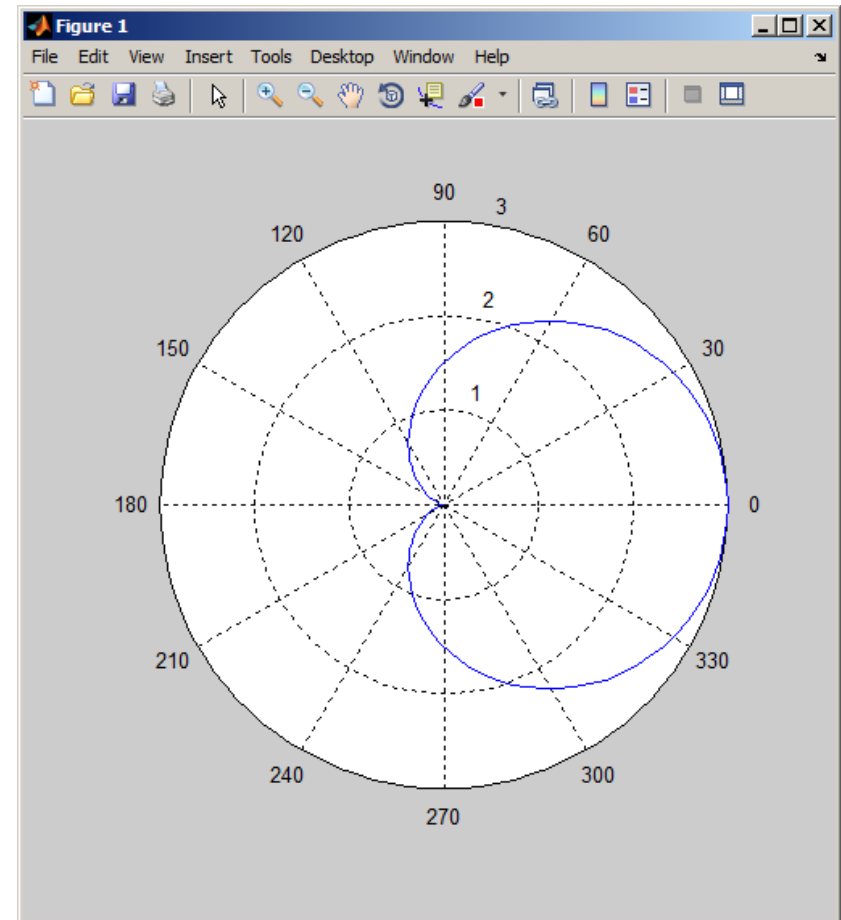
```
subplot(2,2,4)  
loglog(x,y);  
grid on;  
title('loglog');
```



4. Grafische Darstellung – 4.4 2-dim Graphen

Polares Diagramm:

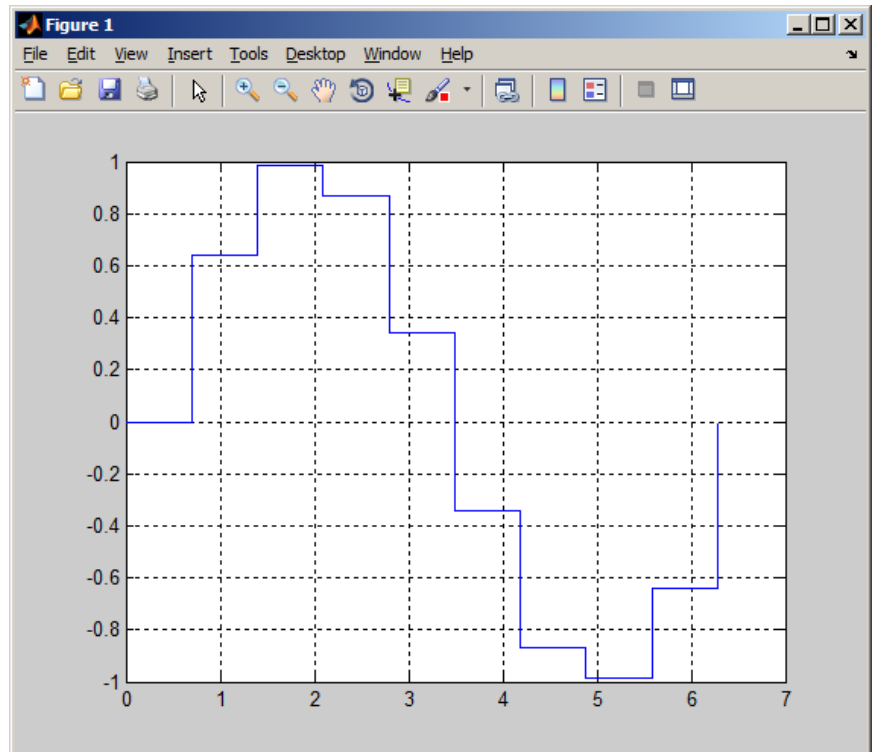
```
phi=linspace(0,2*pi);  
r=3/2*(1+cos(phi));  
polar(phi,r);
```



4. Grafische Darstellung – 4.4 2-dim Graphen

Stufenförmige Kurven (Stairs):

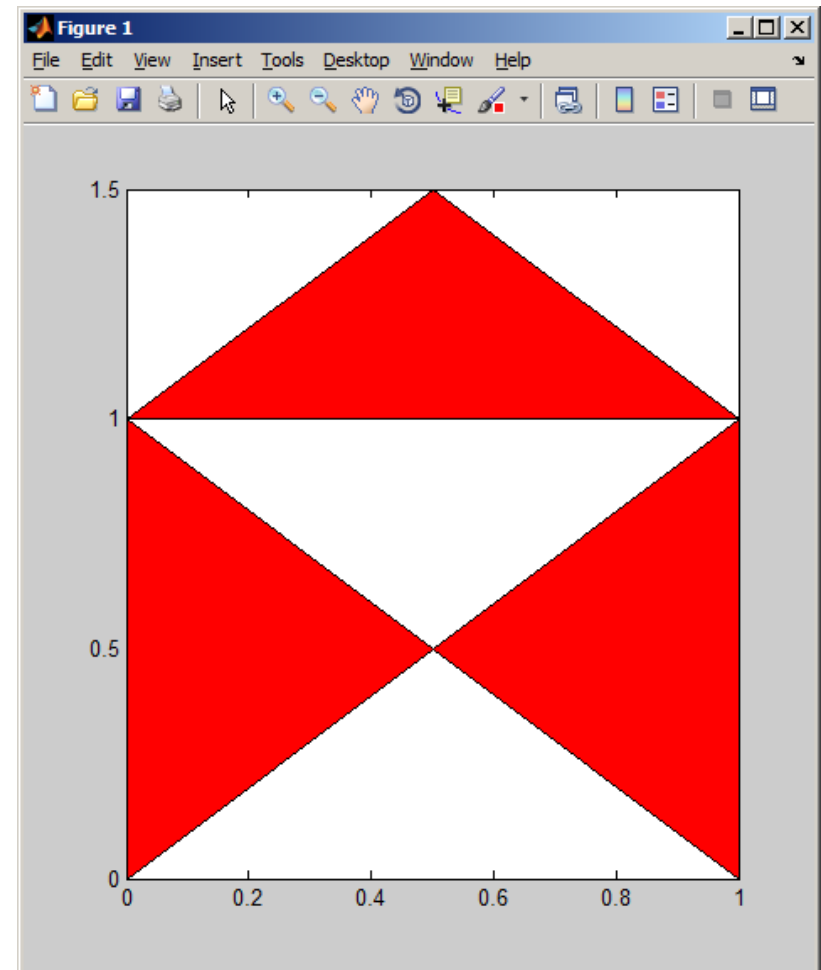
```
x = linspace(0,2*pi,10);  
y = sin(x);  
stairs(x,y);  
grid on;
```



4. Grafische Darstellung – 4.4 2-dim Graphen

Gefüllte Flächen:

```
x = [0 0 0.5 1 0 1 0 1 1];  
y = [0 1 1.5 1 1 0 0 1 0];  
  
fill(x,y,'r');
```



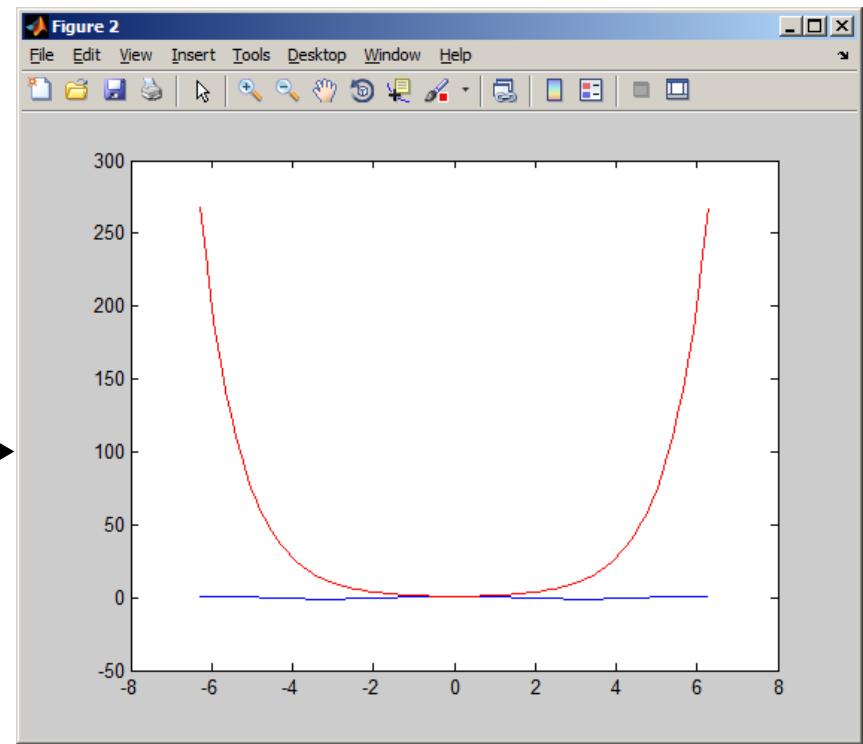
4. Grafische Darstellung – 4.4 2-dim Graphen

Gemischte Verwendung der Grafikbefehle:

Folgendes funktioniert *nicht*:

```
figure(2)
x=linspace(-2*pi,2*pi);
y1=cos(x);
y2=cosh(x);
plot(x,y1,'b')
hold on;
semilogy(x,y2,'r')
```

Der `semilogy`-Befehl wird auf die ursprüngliche lineare Skalierung des `plot`-Befehls angewendet, da ein ‚Axes‘ nur ein Art von Skalierung haben kann.



4. Grafische Darstellung – 4.4 2-dim Graphen

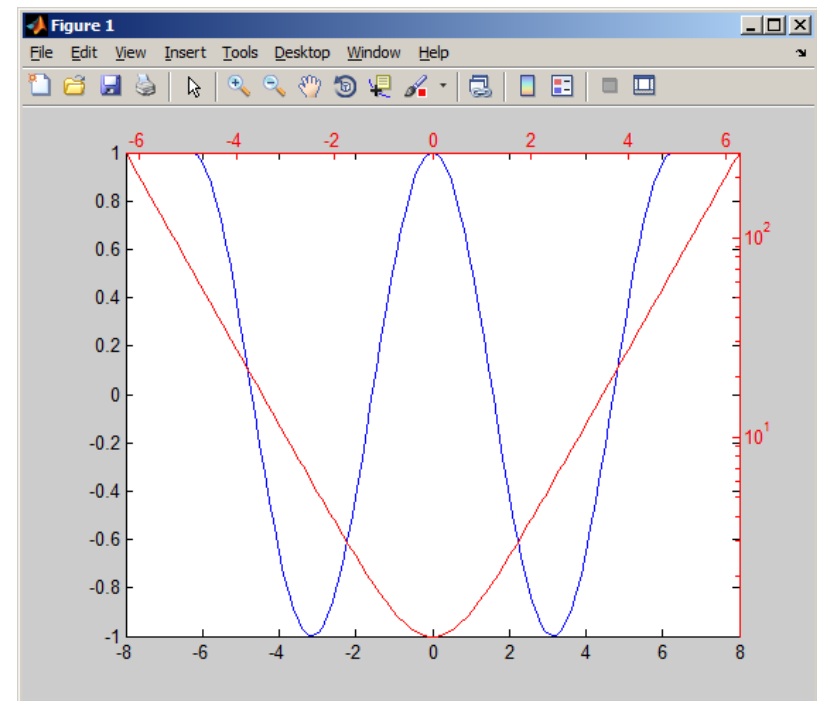
Zum Mischen von unterschiedlichen Skalierungen können aber mehrere „Axes“ übereinander gelegt werden:

```
figure(1)
x=linspace(-2*pi,2*pi);
y1=cos(x);
y2=cosh(x);

plot(x,y1,'b')
hold on;
ax1 = gca;

ax2 = axes('Position',get(ax1,'Position'),...
           'XAxisLocation','top',...
           'YAxisLocation','right','Color','none',...
           'XColor','r','YColor','r','YScale','log');

line(x,y2,'Parent',ax2,'Color','r')
axis tight
```



4. Grafische Darstellung – 4.4 2-dim Graphen

Wichtige Unterscheidung:

Befehle wie `plot`, `semilogx`, `semilogy`, `loglog` erzeugen ein „Axis“ mit einer entsprechenden Skalierung und zeichnen den gewünschte Polygonzug.

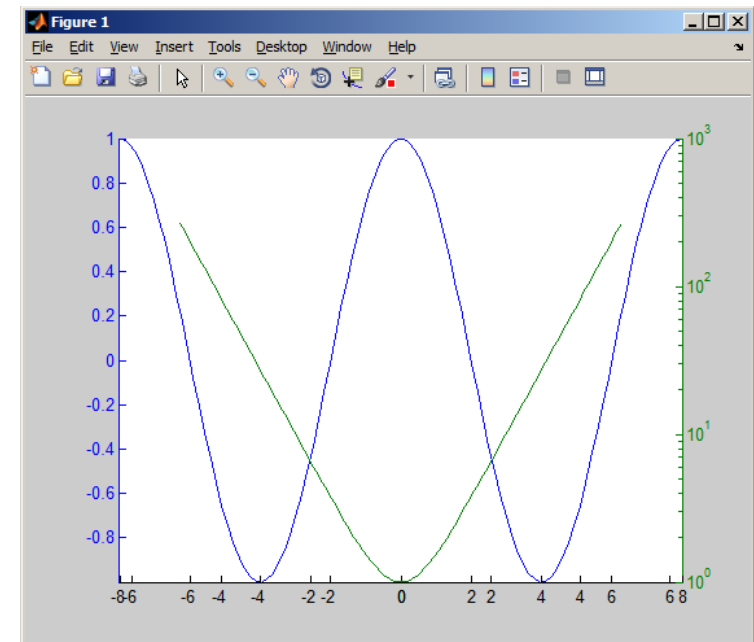
Der `axis`-Befehl erzeugt lediglich ein „Axis“ .

Der `line`-Befehl zeichnet einen Polygonzug in ein bestehendes „Axis“ mit einer beliebigen Skalierung .

4. Grafische Darstellung – 4.4 2-dim Graphen

Auch der plotyy-Befehl unterstützt die Darstellung gemischter Grafiken:

```
x=linspace(-2*pi,2*pi);  
y1=cos(x);  
y2=cosh(x);  
plotyy(x,y1,x,y2,@plot,@semilogy)  
axis tight
```



4. Grafische Darstellung – 4.4 2-dim Graphen

```
>> help graph2d
```

Two dimensional graphs.

Elementary X-Y graphs.

plot	- Linear plot.
loglog	- Log-log scale plot.
semilogx	- Semi-log scale plot.
semilogy	- Semi-log scale plot.
polar	- Polar coordinate plot.
plotyy	- Graphs with y tick labels on the left and right.

Axis control.

axis	- Control axis scaling and appearance.
zoom	- Zoom in and out on a 2-D plot.
grid	- Grid lines.
box	- Axis box.
rbbox	- Rubberband box.
hold	- Hold current graph.



4. Grafische Darstellung – 4.4 2-dim Graphen

- axes - Create axes in arbitrary positions.
- subplot - Create axes in tiled positions.

Graph annotation.

```

plottedit  - Tools for editing and annotating plots.
title      - Graph title.
xlabel     - X-axis label.
ylabel     - Y-axis label.
texlabel   - Produces the TeX format from a character string.
text       - Text annotation.
gtext      - Place text with mouse.

```

Hardcopy and printing.

```
print      - Print graph or Simulink system; or save graph to
                                                    M-file.
printopt   - Printer defaults.
orient     - Set paper orientation.
```

See also `graph3d`, `specgraph`.

4. Grafische Darstellung – 4.5 Diagramme

Diagrammfunktion:

<code>bar</code>	→ Balkendiagramm (vertikal)
<code>barh</code>	→ Balkendiagramm (horizontal)
<code>bar3</code>	→ Balkendiagramm 3-dimensional (vertikal)
<code>bar</code>	→ Balkendiagramm 3-dimensional (horizontal)
<code>pie</code>	→ Tortendiagramm
<code>pie3</code>	→ Tortendiagramm 3-dimensional
<code>stem</code>	→ Diskrete Daten
<code>stem3</code>	→ Diskrete Daten 3-dimensional
<code>hist</code>	→ Histogramm

4. Grafische Darstellung – 4.5 Diagramme

Balkendiagramm

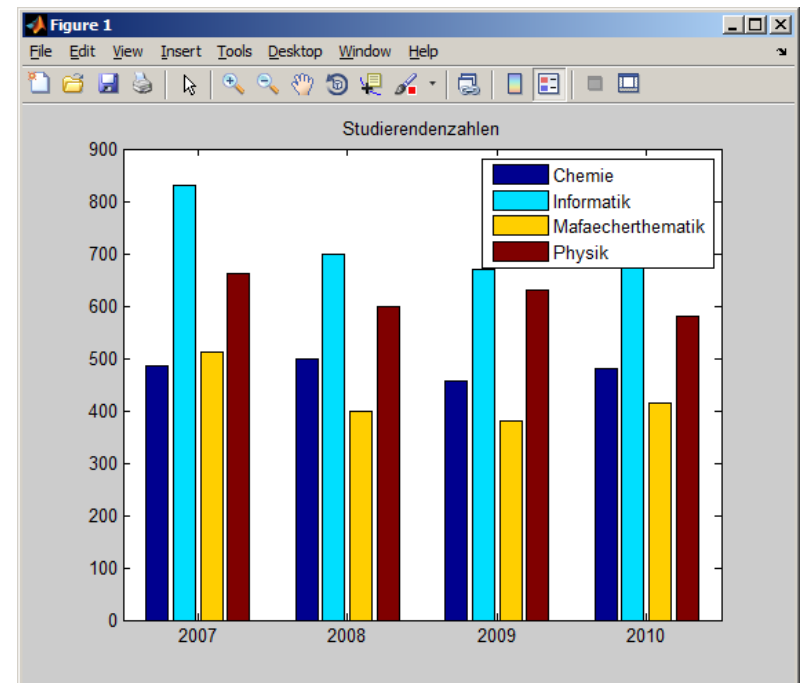
```
figure(1);

studierende=[485 831 513 661;...
             500 700 400 600;...
             456 671 380 630;...
             480 720 414 580];

jahre = [2007 2008 2009 2010];
faecher={'Chemie','Informatik',...
        'Mafaecherthematik','Physik'};

bar(jahre,studierende)

legend(faecher)
title('Studierendenzahlen')
```



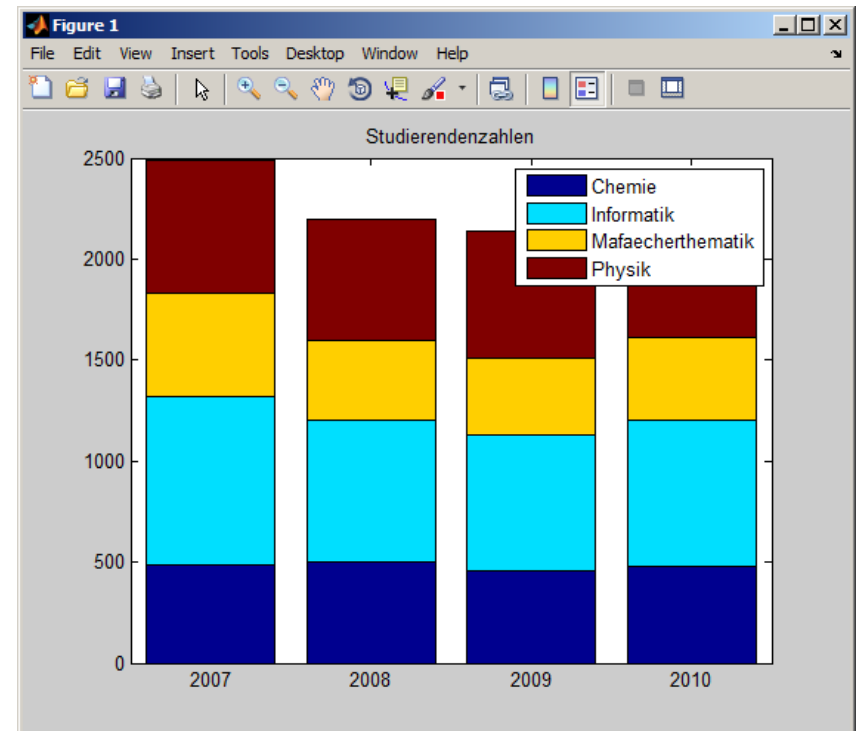
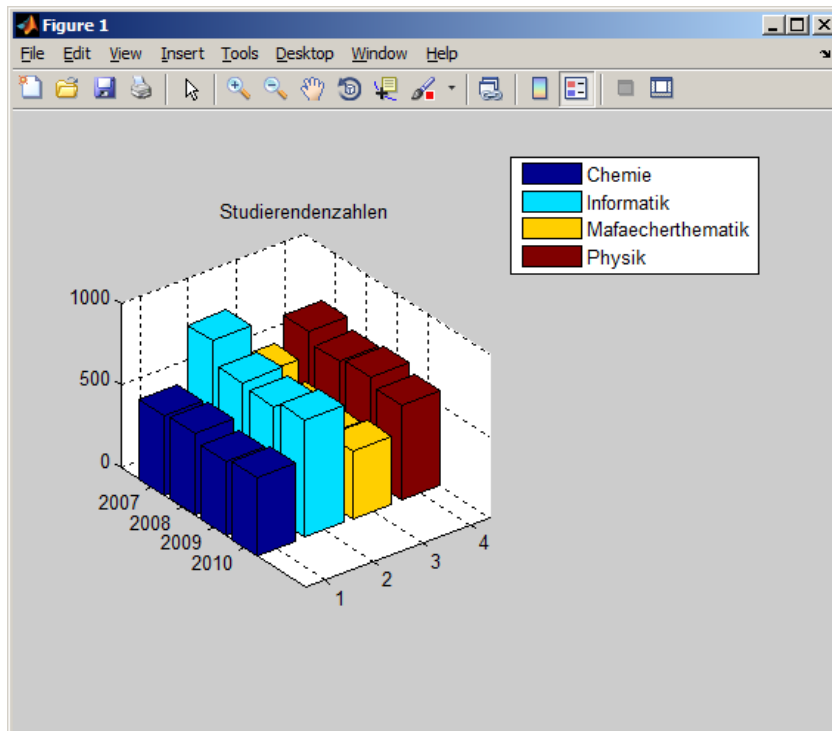
4. Grafische Darstellung – 4.5 Diagramme

oder

```
...
bar3(jahre,studierende)
...
```

oder

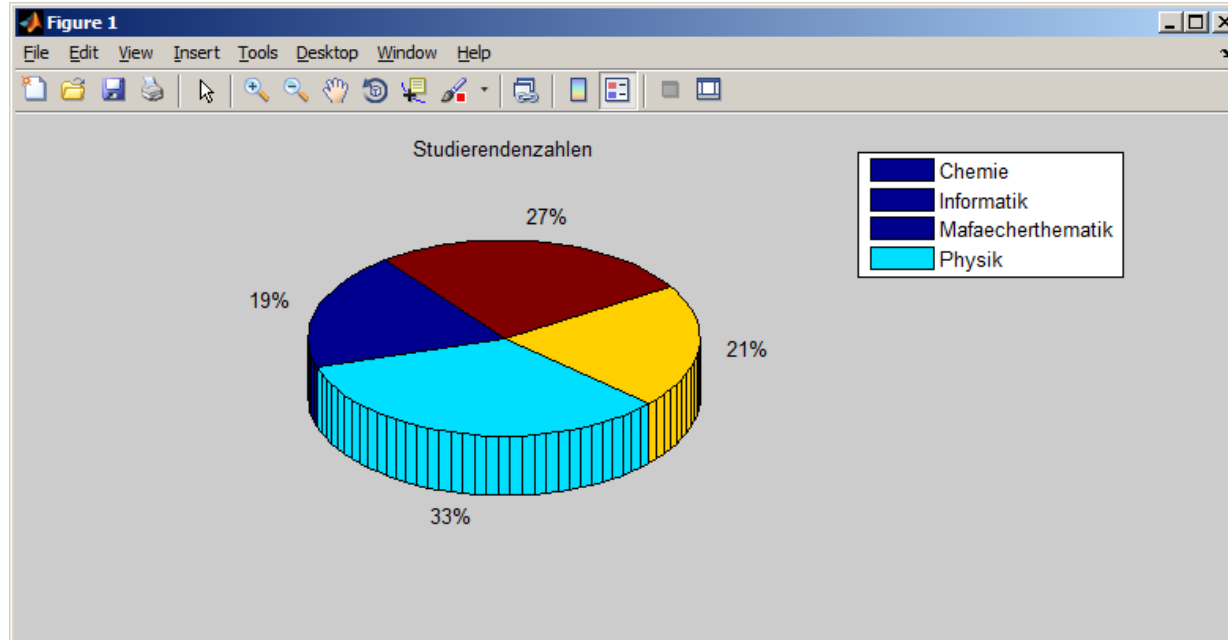
```
...
bar(jahre,studierende,'stacked')
...
```



4. Grafische Darstellung – 4.5 Diagramme

Tortengrafik:

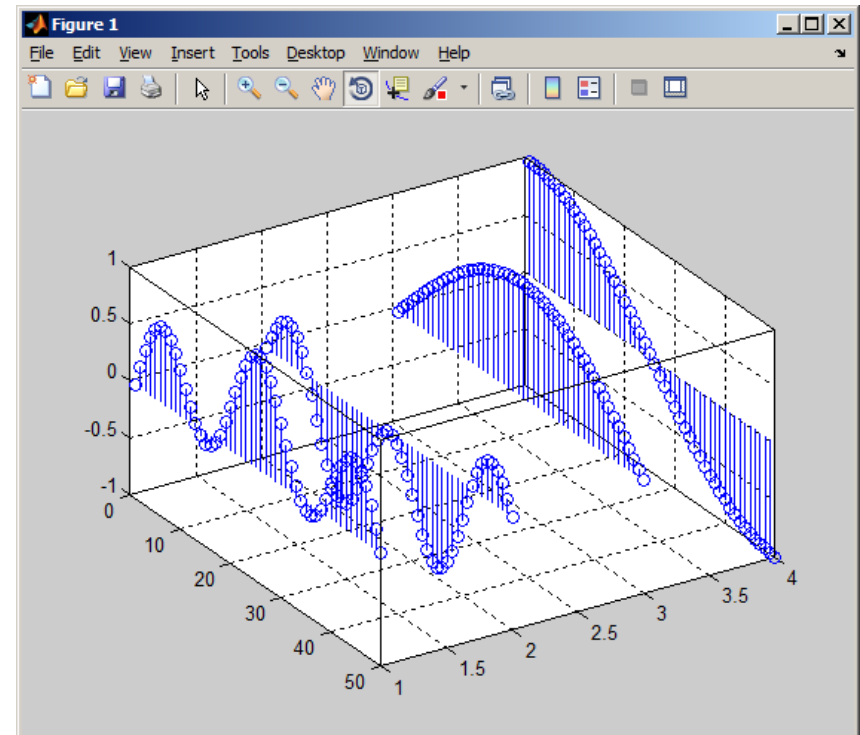
```
...  
pie3(studierende(1,:))  
  
legend(faecher)  
title('Studierendenzahlen')
```



4. Grafische Darstellung – 4.5 Diagramme

Darstellung diskreter Daten:

```
x=linspace(0,pi,50);  
y = [sin(3*x).*cos(2*x);...  
     cos(3*x).*sin(2*x);...  
     sin(x);...  
     cos(x)];  
stem3(y)
```



4. Grafische Darstellung – 4.6 3-dim Grafiken

Dreidimensionale Grafiken:

<code>plot3</code>	→ Polygonzug im 3-dim. Raum
<code>mesh</code>	→ 3-dim. Netz
<code>surf</code>	→ 3-dim. Fläche
<code>surfl</code>	→ 3-dim. Fläche mit Licht

Wichtige Formatierungsbefehle:

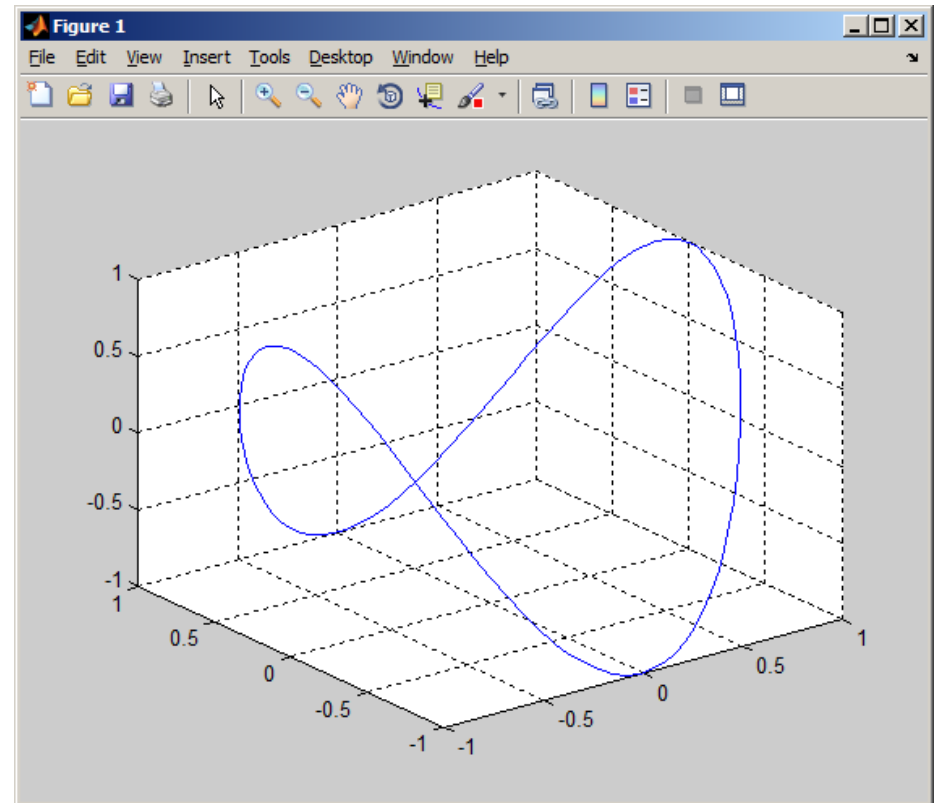
<code>colormap</code>	→ Definition der Darstellungsfarben
<code>shading</code>	→ Einstellen der Darstellung der Kanten und Flächen
<code>hidden</code>	→ Einstellen der Liniendarstellung
<code>alpha</code>	→ Einstellen der Transparenz
<code>colorbar</code>	→ Anzeige der Farbpalette
<code>lighting</code>	→ Einstellen der Beleuchtung



4. Grafische Darstellung – 4.6 3-dim Grafiken

Dreidimensionaler Polygonzug:

```
figure(1);  
t=linspace(0,2*pi);  
x=cos(t);  
y=sin(t);  
z=cos(2*t);  
plot3(x,y,z);  
grid on;
```

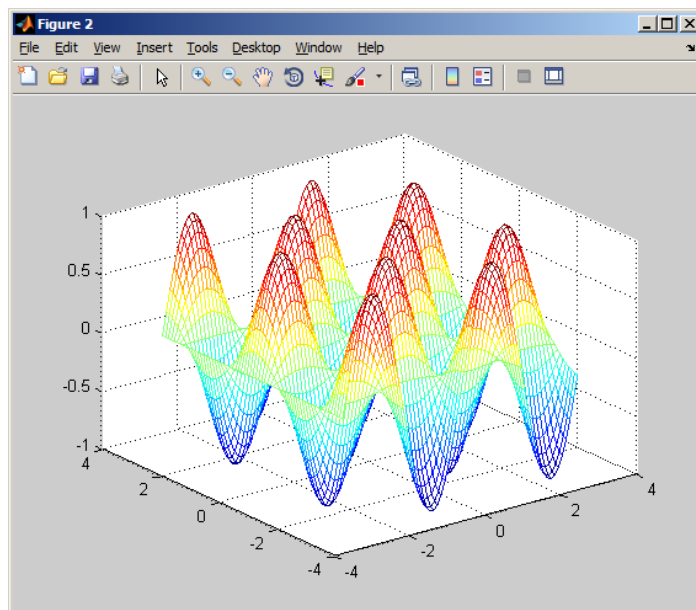


4. Grafische Darstellung – 4.6 3-dim Grafiken

Gitternetzdarstellung

...einer Kennfläche

```
figure(2);  
[x,y]=meshgrid(-pi:.1:pi,-pi:.1:pi);  
z=sin(2*x).*cos(2*y);  
mesh(x,y,z);
```



Hinweis:

x, y und z sind hier Matrizen gleicher Größe, wobei x und y in diesem Fall eine quadratische (Definitions-)Fläche mit äquidistanten Stützstellen beschreiben.

4. Grafische Darstellung – 4.6 3-dim Grafiken

Der `meshgrid`-Befehl:

```
>> [x,y]=meshgrid(0:1:5,0:1:5);
```

```
>> x
```

x =

0	1	2	3	4	5
0	1	2	3	4	5
0	1	2	3	4	5
0	1	2	3	4	5
0	1	2	3	4	5
0	1	2	3	4	5

} 2x2-Array in x-Richtung steigend

```
>> y
```

y =

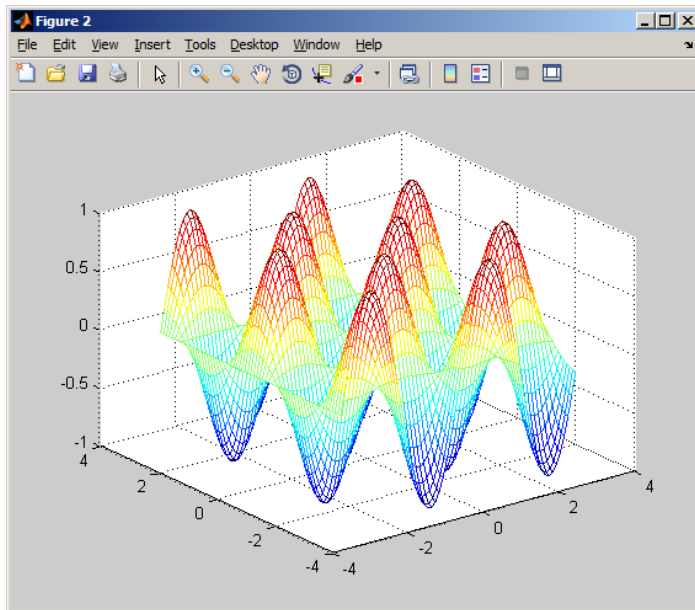
0	0	0	0	0	0
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4
5	5	5	5	5	5

} 2x2-Array in y-Richtung steigend

4. Grafische Darstellung – 4.6 3-dim Grafiken

Alternative funktioniert auch:

```
figure(3);  
x = (-pi:.1:pi);  
y = (-pi:.1:pi);  
z=transpose(sin(2*x))*cos(2*y);  
mesh(x,y,z);
```



Hinweis:

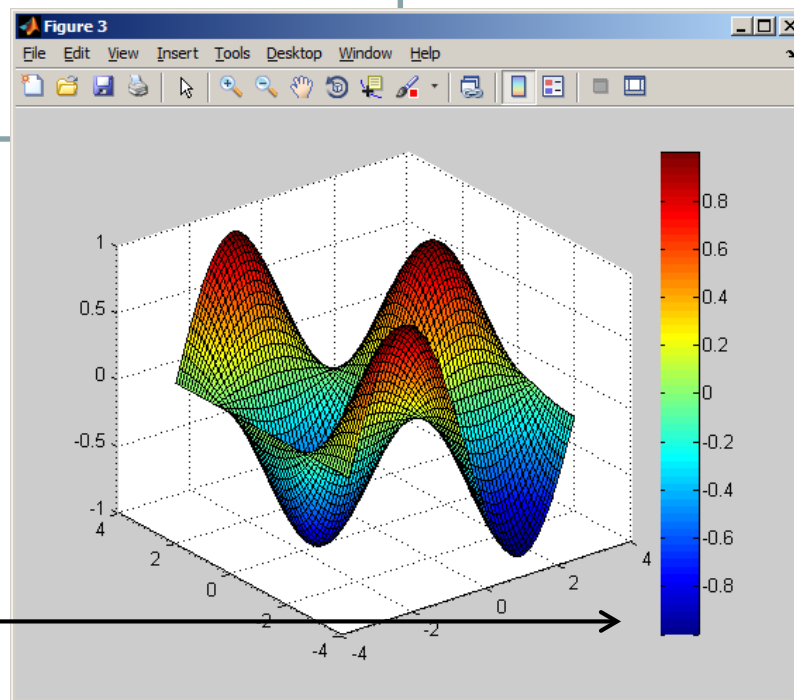
Hier sind x und y Vektoren und z ist weiterhin Matrix mit zu x und y passender Größe.

4. Grafische Darstellung – 4.6 3-dim Grafiken

Gitternetzdarstellung

...einer Kennfläche

```
figure(3);  
[x,y]=meshgrid(-pi:.1:pi,-pi:.1:pi);  
z=sin(x).*cos(y);  
surf(x,y,z);  
colorbar;
```



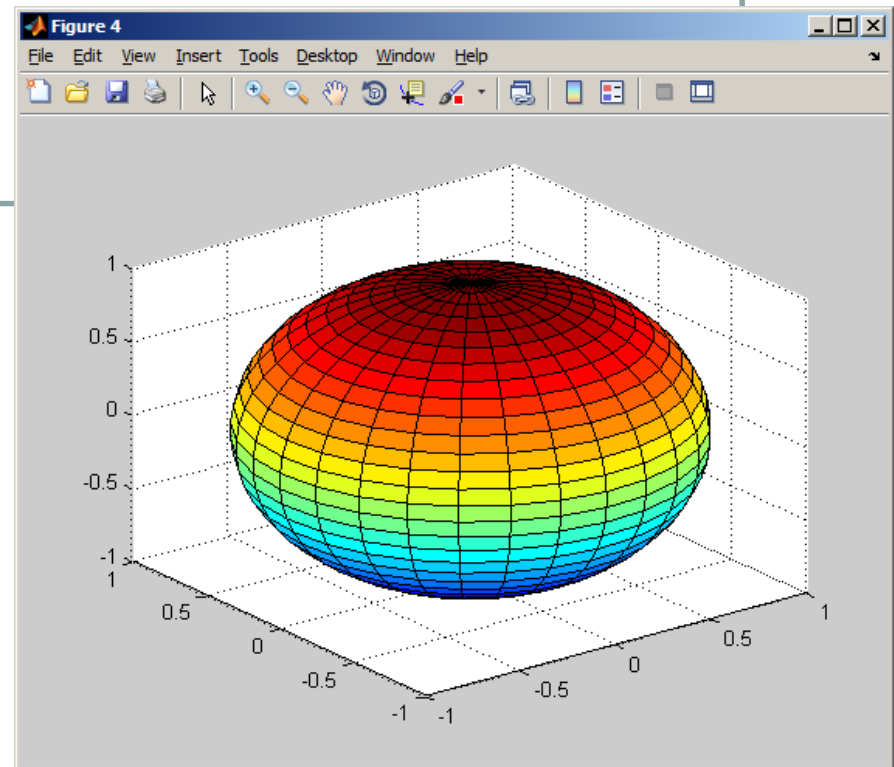
4. Grafische Darstellung – 4.6 3-dim Grafiken

Die durch x y und z im 3-dim. Raum aufgespannte Fläche muss nicht unbedingt eine Graph im mathematischen Sinne sein:

```
figure(4);  
[p,t]=meshgrid(linspace(-pi,pi,30),linspace(0,pi,30));  
x=cos(p).*sin(t);  
y=sin(p).*sin(t);  
z=cos(t);  
surf(x,y,z);
```

Hinweis:

p und t definieren hier eine quadratische (Definitions-)Fläche äquidistanter Stützstellen, aufgrund derer der Kugelmantel (beschrieben durch Matrizen x , y und z) wie die Abwicklung eines Globus berechnet wird.



4. Grafische Darstellung – 4.6 3-dim Grafiken

Formatierungsbefehle:

```
colormap(map)
```

→ definiert in eine Farbpalette durch eine nx3-Matrix mit Werten zwischen 0 und 1 wobei die Werte in den Spalten die Anteil im Sinne des RGB-Farbsystems definieren. Die n Zeilen verteilen sich gleichmäßig auf den Zielbereich der Darstellung.

Bsp.

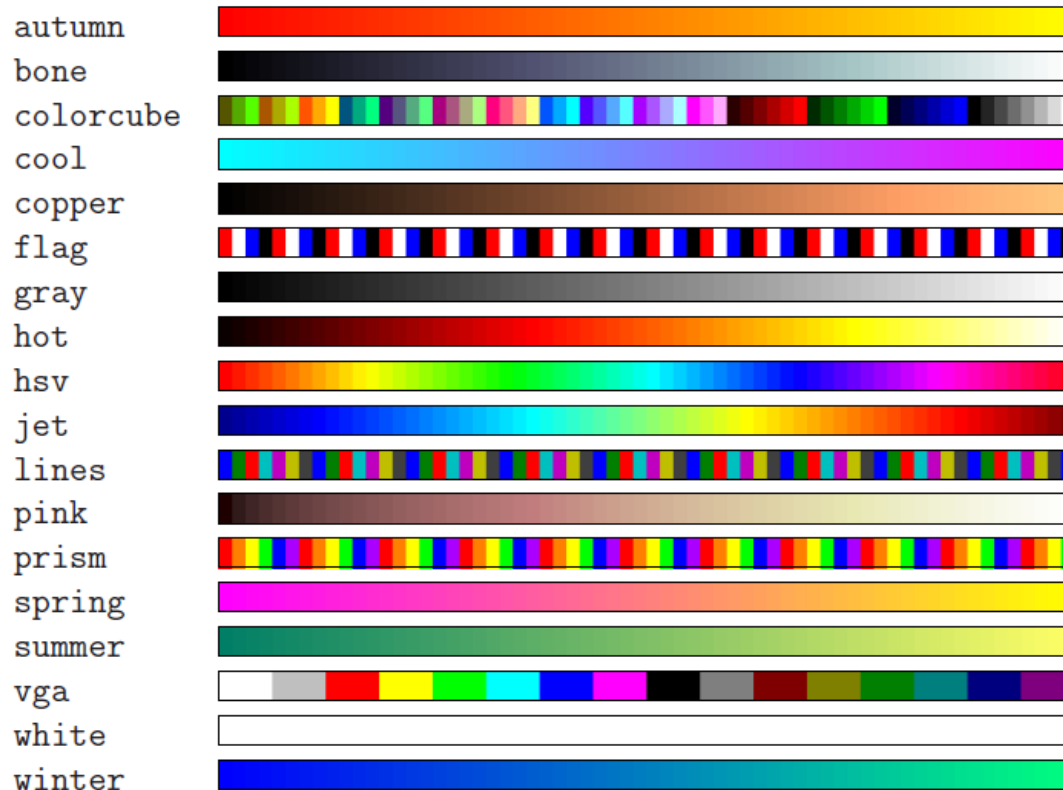
```
Map = [1 1 0; ...  
       0.8 0.8 0; ...  
       0.6 0.6 0; ...  
       0.4 0.4 0; ...  
       0.2 0.2 0];
```

(Verschiedene Gelbtöne)

Hinweis: Die „Colormap“ verändert auch die Farbdarstellung der Diagramme

4. Grafische Darstellung – 4.6 3-dim Grafiken

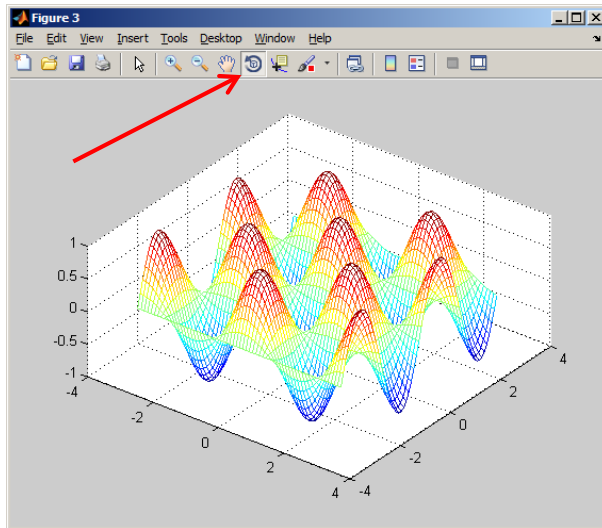
Es gibt zahlreiche vordefiniert Farbpaletten:



Aufruf: `gray` oder `summer(<numbl>)`
(`<numbl>` ist die Anzahl der Zeilen)

4. Grafische Darstellung – 4.6 3-dim Grafiken

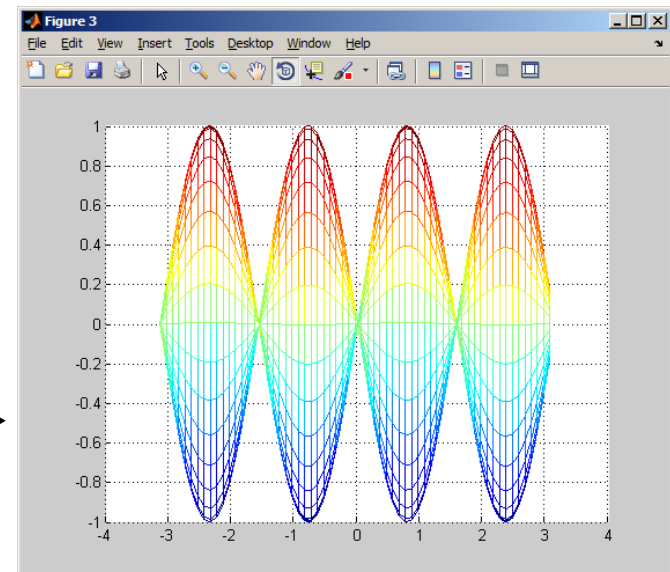
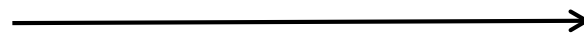
Das Drehen einer d3-dim. Grafik kann interaktiv im Grafikfenster erfolgen:



oder mit dem Befehl

```
view(<azimuth>,<elevation>)
```

```
view(90,0);
```



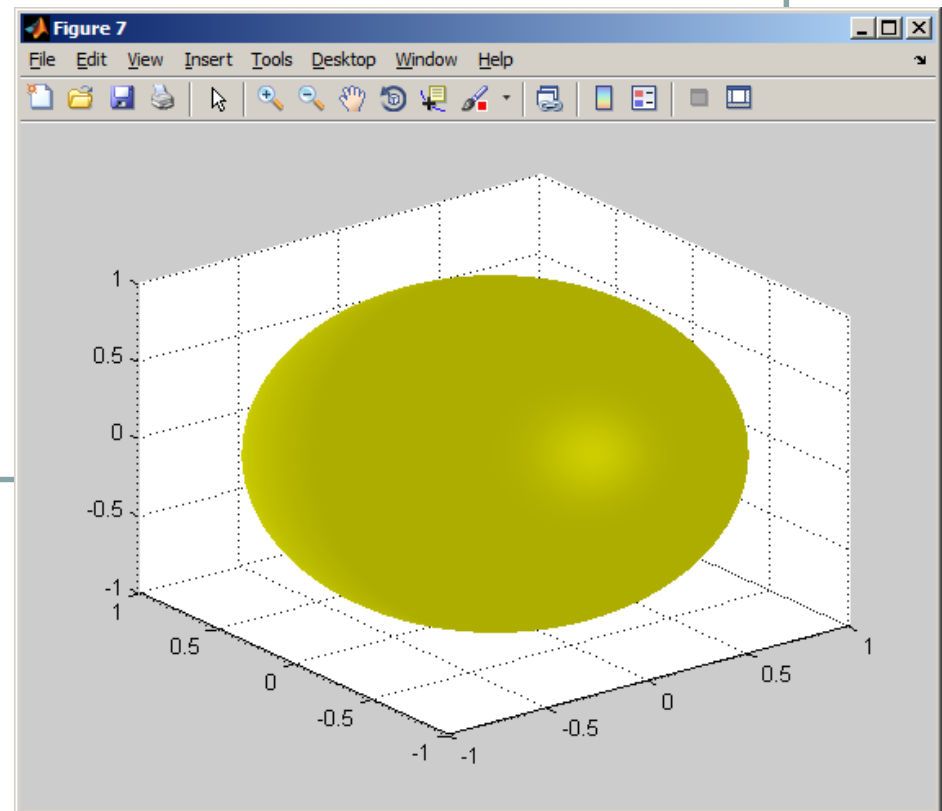
4. Grafische Darstellung – 4.6 3-dim Grafiken

Der surf1-Befehl erzeugt plastische 3-dim. Darstellungen:

```
figure(7);  
[p,t]=meshgrid(linspace(-pi,pi,100),linspace(0,pi,100));  
x=cos(p).*sin(t);  
y=sin(p).*sin(t);  
z=cos(t);  
surf1(x,y,z);  
cm = gray(1000);  
cm(:,3)= 0 * cm(:,3);  
cm = cm(500:1000,:);  
colormap(cm);  
shading interp;
```

Hinweis:

Die „Colormap“ definiert hier die der „Beleuchtungsstärke“ zugehörige Farbe



4. Grafische Darstellung – 4.6 3-dim Grafiken

Befehle zur Steuerung der Beleuchtung:

```
lighting, material, camlight, lightangle
```

(Siehe Matlab-Hilfe)

4. Grafische Darstellung – 4.7 Schnittgrafiken

Darstellung von Schnittbildern:

Dreidimensional:

<code>contour</code>	→ Grafik mit Höhenlinien
<code>contourf</code>	→ Grafik mit Höhenlinien und Farbflächen
<code>contour3</code>	→ 3-dim. Darstellung von Höhenlinien
<code>meshc</code>	→ Kombinierte Darstellung mit 3-dim. Gitter und Höhenlinien
<code>surfc</code>	→ Kombinierte Darstellung mit 3-dim. Fläche und Höhenlinien
<code>clabel</code>	→ Beschriftung von Höhenlinien
<code>imagesc</code>	→ Farbebenenendarstellung

Vierdimensional:

<code>slice</code>	→ Funktionswerte in Schnitten eines 4-dim- Kennfeldes
<code>contourslice</code>	→ Höhenlinien in Schnitten eines 4-dim- Kennfeldes

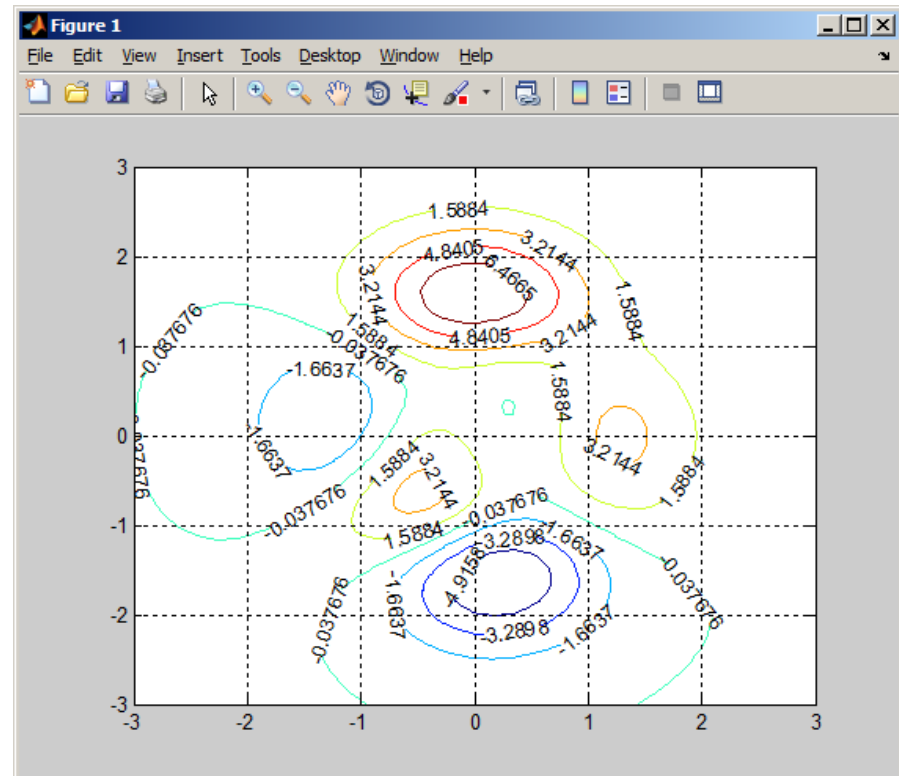


4. Grafische Darstellung – 4.7 Schnittgrafiken

Darstellung von Höhenlinien:

```
figure(1)  
[X,Y,Z]=peaks(100);  
[c,h]=contour(X,Y,Z,8);  
clabel(c,h);  
grid on;
```

8 Linien

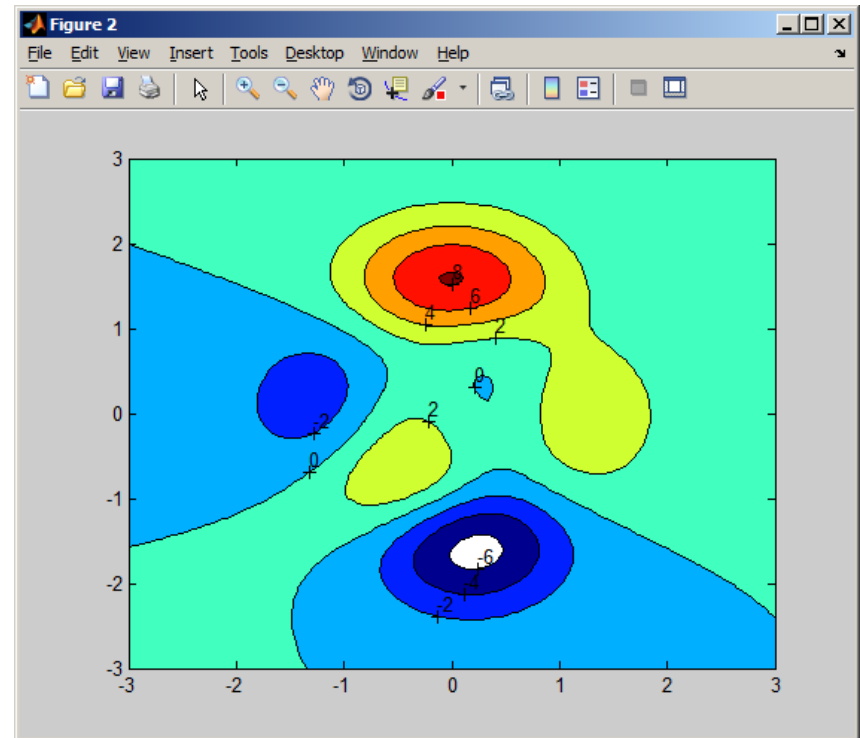


4. Grafische Darstellung – 4.7 Schnittgrafiken

Darstellung von Höhenlinien mit Farbflächen:

```
figure(2)  
[X,Y,Z]=peaks(100);  
[c,h]=contourf(X,Y,Z,-6:2:8);  
clabel(c);
```

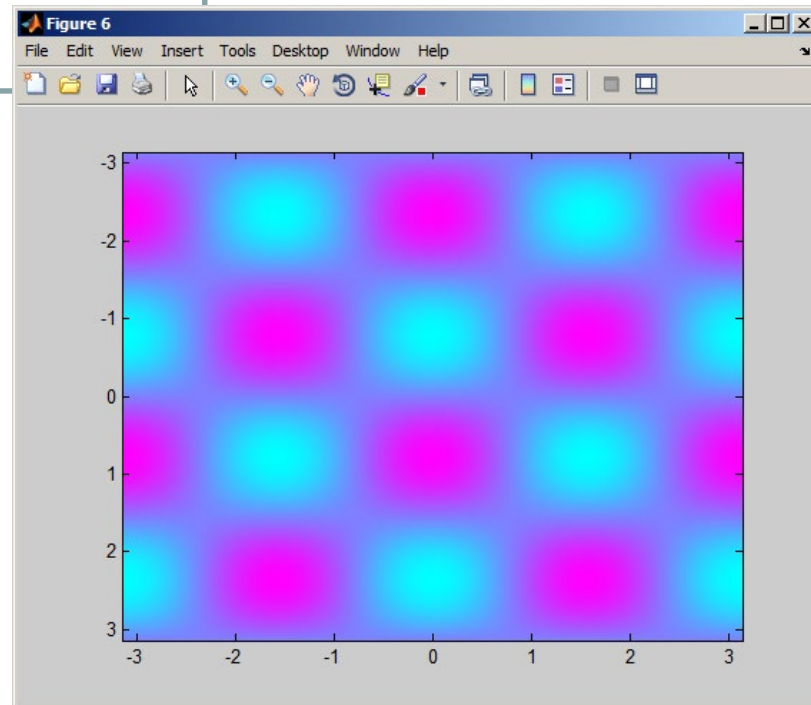
Definition der Höhenlinie



4. Grafische Darstellung – 4.7 Schnittgrafiken

Farbebenenendarstellung:

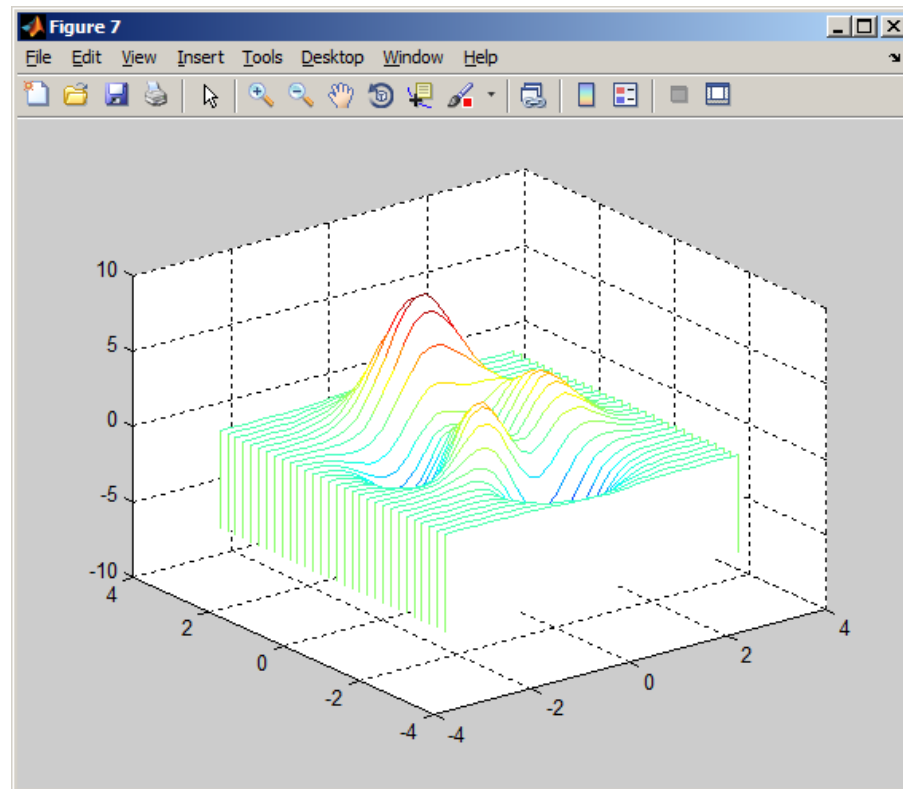
```
figure(6);  
x = (-pi:.01:pi);  
y = (-pi:.01:pi);  
z=transpose(sin(2*x))*cos(2*y);  
colormap(cool(100));  
imagesc(x,y,z);
```



4. Grafische Darstellung – 4.7 Schnittgrafiken

Wasserfalldiagramm:

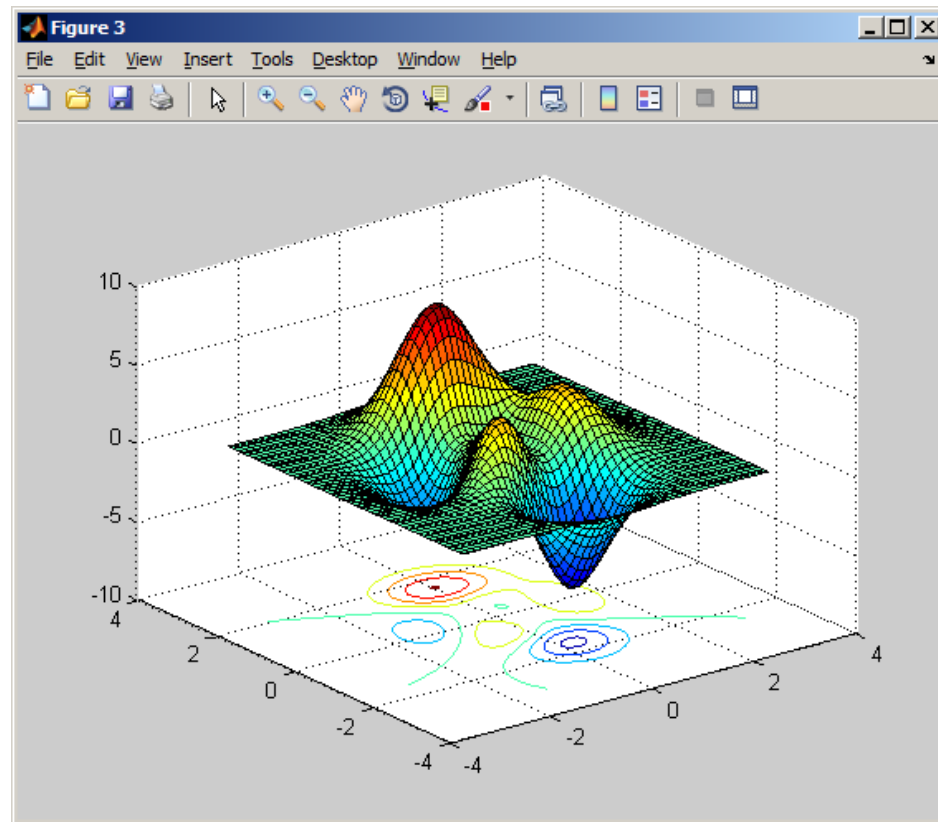
```
figure(7)  
[X,Y,Z] = peaks(30);  
waterfall(X,Y,Z);
```



4. Grafische Darstellung – 4.7 Schnittgrafiken

Kombinierte Grafik:

```
figure(3)  
[X,Y,Z]=peaks;  
surfc(X,Y,Z);
```



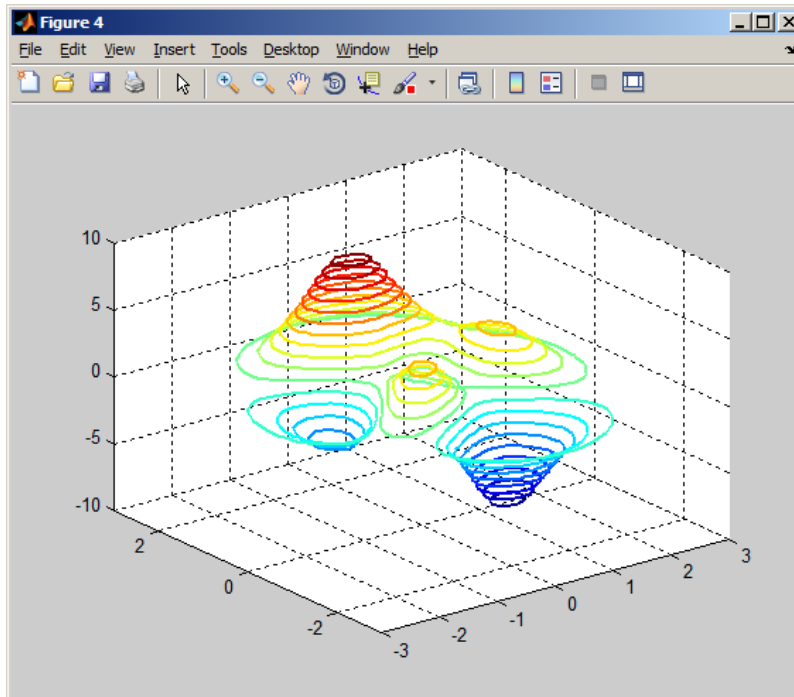
4. Grafische Darstellung – 4.7 Schnittgrafiken

3-dim. Höhenlinien:

```
figure(4)
[X,Y,Z]=peaks;
[h,handles]= contour3(X,Y,Z,20);
set(handles,'LineWidth',2)
```

Hinweis:

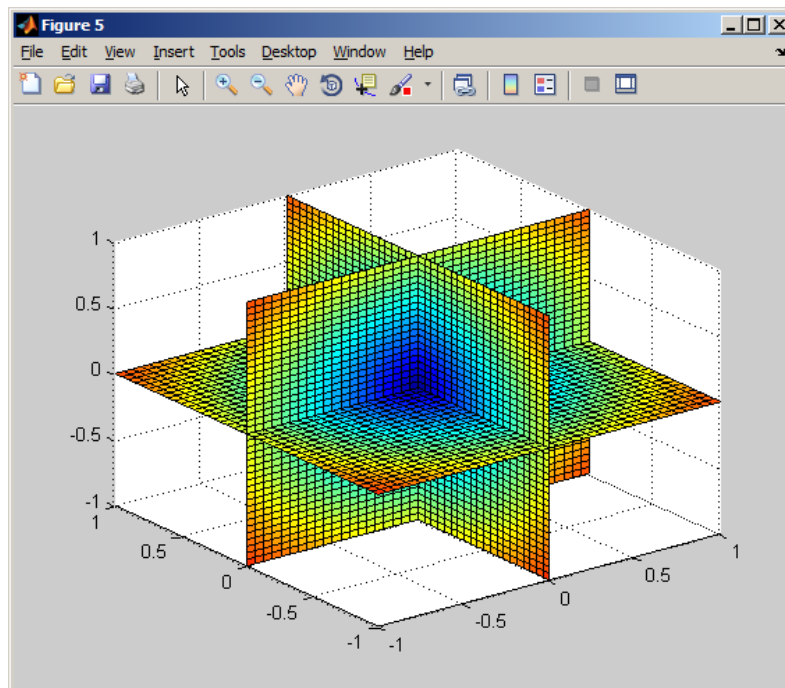
Die Linienstärke wird hier über den `set`-Befehl verändert wobei `handles` ein Vektor mit „Handles“ aller Höhenlinien ist.



4. Grafische Darstellung – 4.7 Schnittgrafiken

4-dim. Schnittgrafik:

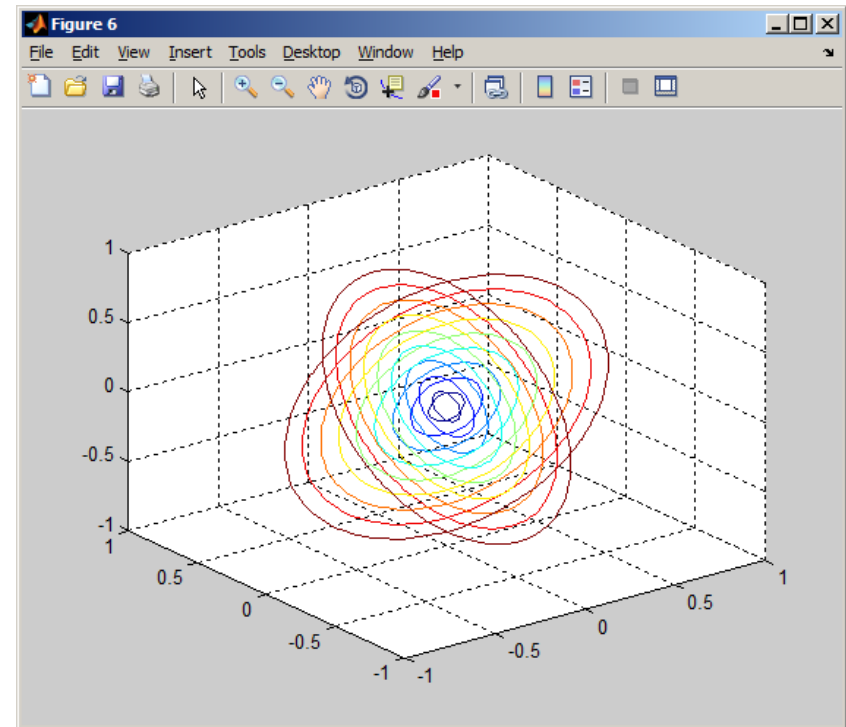
```
figure(5);  
[X,Y,Z]=meshgrid(-1:.05:1);  
V=sqrt(X.^2+Y.^2+Z.^2);  
slice(X,Y,Z,V,0,0,0);
```



4. Grafische Darstellung – 4.7 Schnittgrafiken

Höhenlinien bei 3-dim. Schnitten:

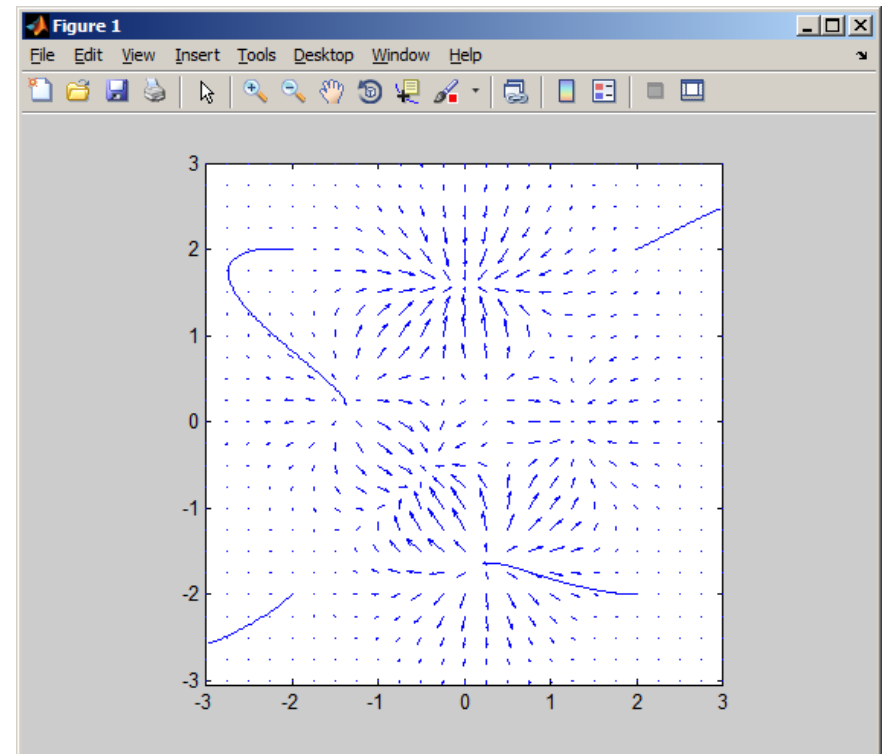
```
figure(6);  
[X,Y,Z]=meshgrid(-1:.05:1);  
V=sqrt(X.^2+Y.^2+Z.^2);  
V(V>1)=nan;  
contourslice(X,Y,Z,V,0,0,[])  
view(3)  
grid on;
```



4. Grafische Darstellung – 4.8 Spezielle Grafiken

2-dim. Vektorfeldern:

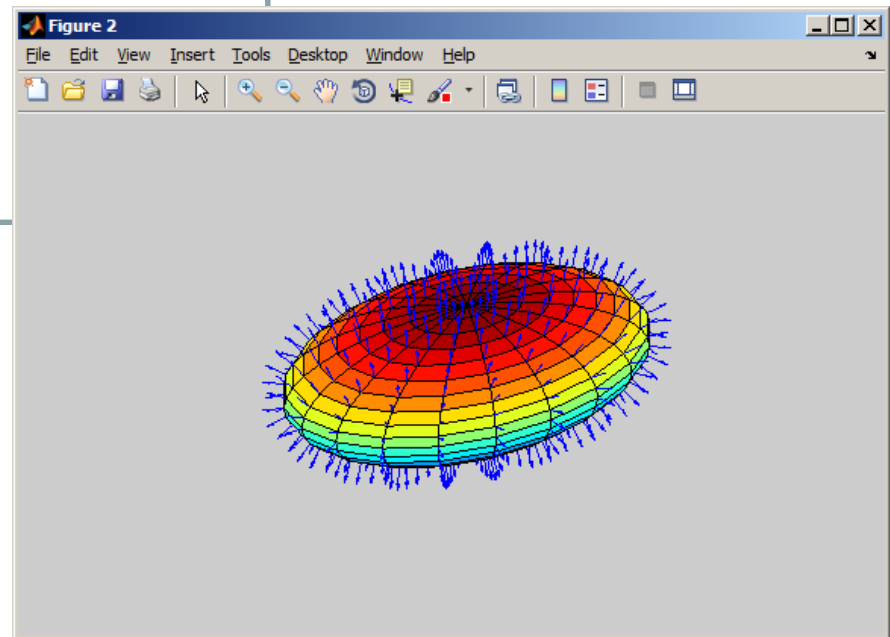
```
figure(1);  
[xx,yy,zz]=peaks(25);  
hx=xx(1,2)-xx(1,1);  
hy=yy(2,1)-yy(1,1);  
[gx,gy]=gradient(zz,hx,hy);  
quiver(xx,yy,gx,gy)  
streamline(xx,yy,-gx,-gy,...  
[-2 2 2 -2],[-2 -2 2 2])  
axis equal  
axis tight
```



4. Grafische Darstellung – 4.8 Spezielle Grafiken

2-dim. Vektorfeldern:

```
figure(2);  
[xx,yy,zz]=ellipsoid(0,0,0,3,2,1,20);  
[nx,ny,nz]=surfnorm(xx,yy,zz);  
surf(xx,yy,zz)  
hold on  
quiver3(xx,yy,zz,nx,ny,nz)  
axis equal  
axis off
```



4. Grafische Darstellung – 4.9 Darstellen von Funktionen

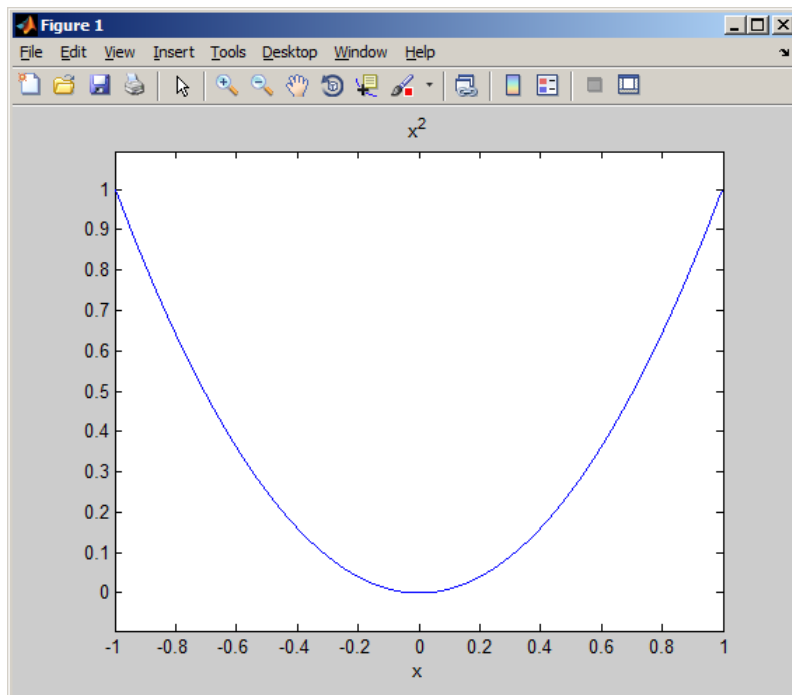
Direktes Darstellen von Funktionen:

<code>ezcontour</code>	→ Funktionsdarstellung mit <code>contour</code>
<code>ezcontourf</code>	→ Funktionsdarstellung mit <code>contourf</code>
<code>ezmesh</code>	→ Funktionsdarstellung mit <code>mesh</code>
<code>ezmeshc</code>	→ Funktionsdarstellung mit <code>meshc</code>
<code>ezplot</code>	→ Funktionsdarstellung mit <code>plot</code>
<code>ezplot3</code>	→ Funktionsdarstellung mit <code>plot3</code>
<code>ezpolar</code>	→ Funktionsdarstellung mit <code>polar</code>
<code>ezsurf</code>	→ Funktionsdarstellung mit <code>surf</code>
<code>ezsurfc</code>	→ Funktionsdarstellung mit <code>surc</code>

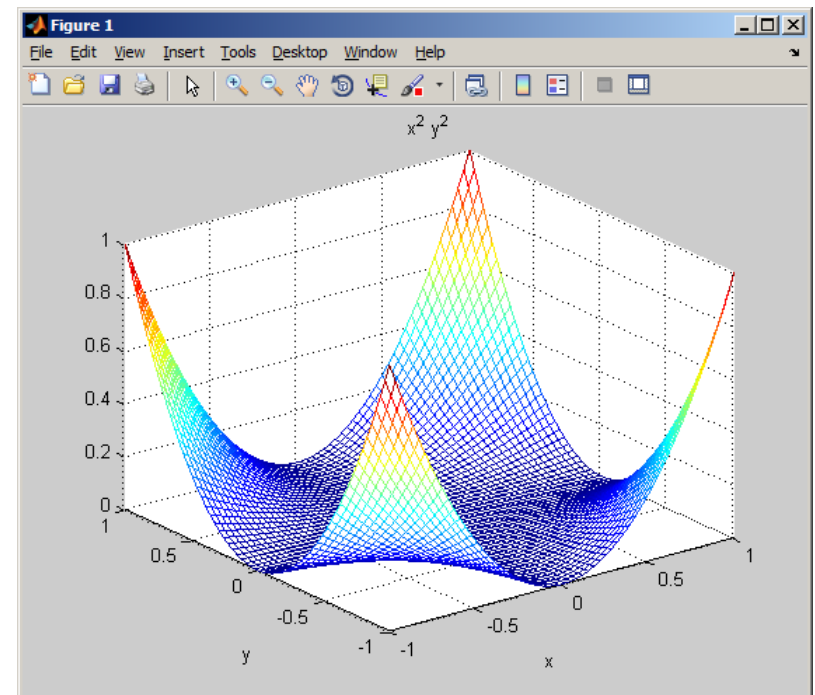


4. Grafische Darstellung – 4.9 Darstellen von Funktionen

```
>> ezplot('x^2', [-1 1]);
```

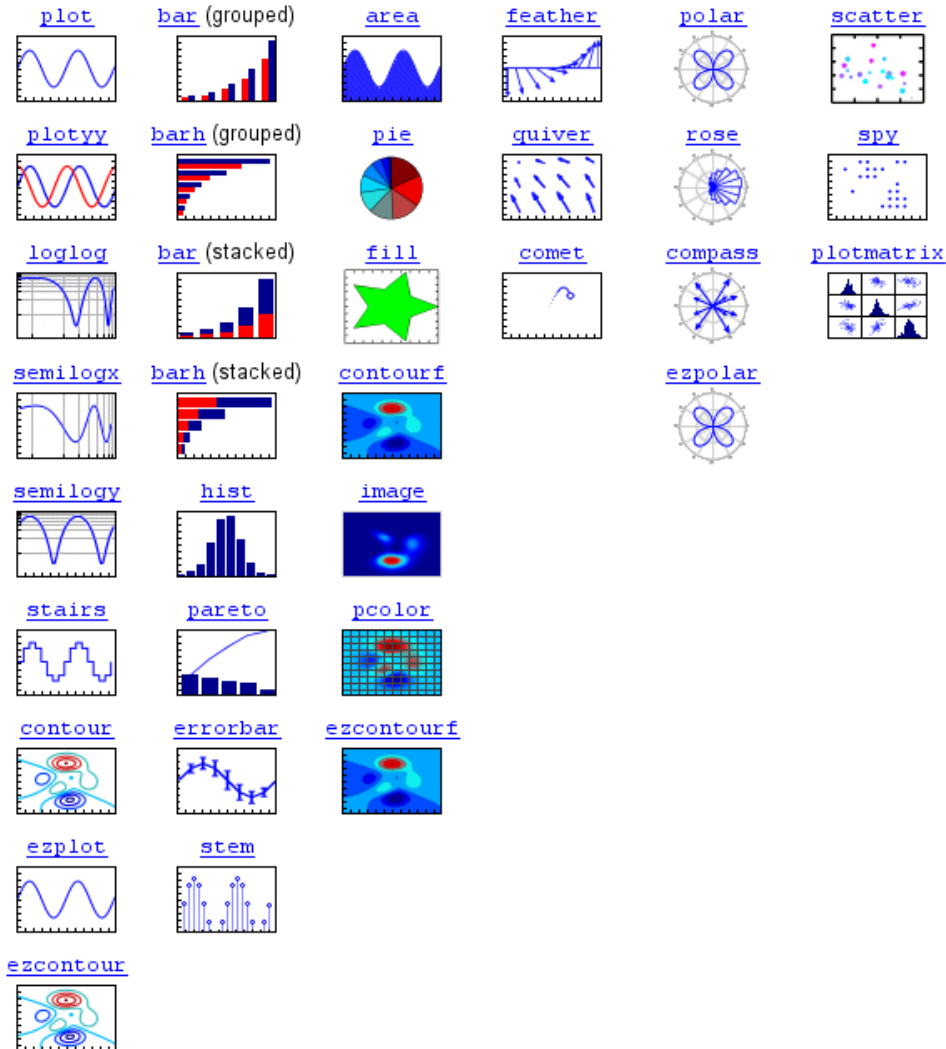


```
>> ezmesh('x^2*y^2', [-1 1], ...  
          [-1 1])
```



4. Grafische Darstellung – 4.10 Übersicht

Übersicht:



4. Grafische Darstellung – 4.10 Übersicht

Übersicht:

Line Graphs

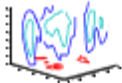
[plot3](#)



[contour3](#)



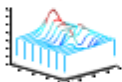
[contourslice](#)



[ezplot3](#)

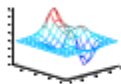


[waterfall](#)

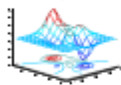


Mesh Graphs and Bar Graphs

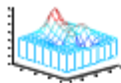
[mesh](#)



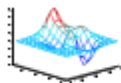
[meshc](#)



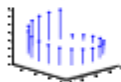
[meshz](#)



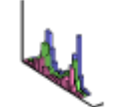
[ezmesh](#)



[stem3](#)



[bar3](#)



Area Graphs and Constructive Objects

[pie3](#)



[fill3](#)



[patch](#)



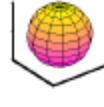
[cylinder](#)



[ellipsoid](#)

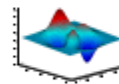


[sphere](#)

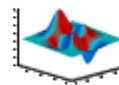


Surface Graphs

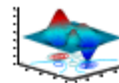
[surf](#)



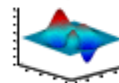
[surfl](#)



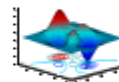
[surfz](#)



[ezsurf](#)

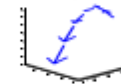


[ezsurfz](#)

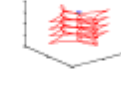


Direction Graphs

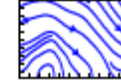
[quiver3](#)



[comet3](#)

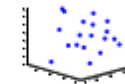


[streamslice](#)

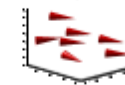


Volumetric Graphs

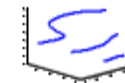
[scatter3](#)



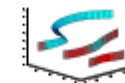
[coneplot](#)



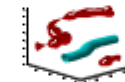
[streamline](#)



[streamribbon](#)



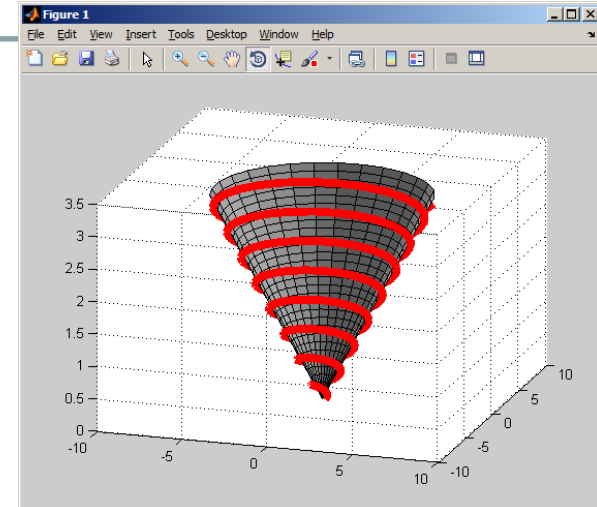
[streamtube](#)



4. Grafische Darstellung – 4.10 Übersicht

Auch Kombinationen von Grafikbefehlen sind möglich:

```
t = linspace(0,2*pi,200);  
u = 8;  
h = 0.5;  
d = 0.2;  
x = sin(t*u).*(t+d);  
y = cos(t*u).*(t+d);  
z = h * t;  
phandle = plot3(x,y,z);  
grid on;  
hold on;  
[p,t]=meshgrid(linspace(0,2*pi,30),linspace(0,2*pi,30));  
x=sin(t).*p;  
y=cos(t).*p;  
z=p*h;  
surf1(x,y,z);  
colormap(gray(30));  
set(phandle,'LineWidth',5,'Color',[1,0,0]);  
hold off;
```



4. Grafische Darstellung – 4.11 GUIs

Erstellen von GUIs:

Die Grafikfenster („figure“) von Matlab können neben Diagrammen („axes“) auch

- Kontrollelemente („uicontrol“)
- (benutzerspezifizierte) Menüs („uimenu“)
- Kontextmenüs („uicontextmenu“)

als „interaktive Elemente“ enthalten.

Mit diesen können in Matlab-Projekten interaktive Oberflächen geschaffen werden, die einem „unbedarften“ Benutzer einen einfachen Umgang mit der Funktion des Projekts bzw. schlicht eine geeignete Benutzerführung ermöglichen.

Beachte:

Neben den programmiertechnischen Fragestellung bedingt die Schaffung eines „guten“ GUI immer auch die Frage nach der Benutzerergonomie. Diese erfordert sowohl eine sinnvollen Strukturierung als auch eine Analyse der Benutzerszenarien (Use cases).

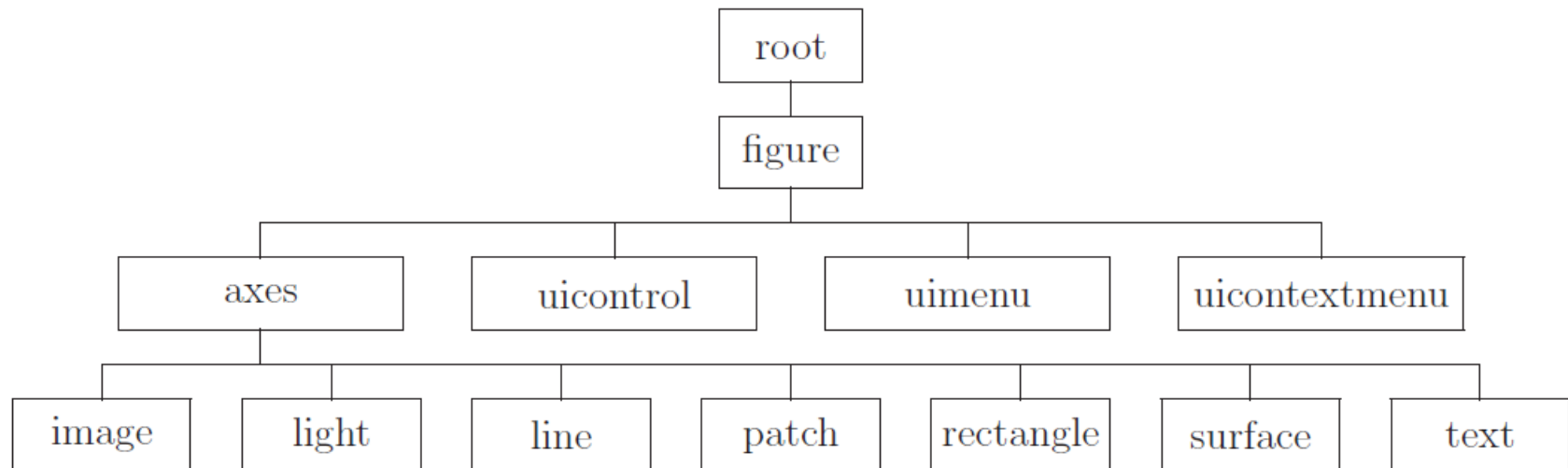
4. Grafische Darstellung – 4.11 GUIs

Eigenschaften von interaktiven Elementen:

- Wie auch die Grafikfenster und Diagramme werden die interaktiven Elemente über „Handles“ verwaltet. Eigenschaften können mit dem `get`-Befehl abgefragt und mit dem `set`-Befehl verändert werden.
- Den interaktiven Elementen sind als Eigenschaft auch sind sogenannte „Callback Functions“ oder kurz „Callbacks“ zugeordnet. Dabei handelt es sich um als String definierte“ Matlab-Anweisungen oder Funktionsaufrufe, die als Reaktion auf ein „Ereignis“ ausgeführt werden.
- Ereignisse sind z. B. das Betätigen eines Kontrollelements , die Auswahl eines Menüs oder aber auch das Öffnen und Schließen von Elementen („OpenCallback“, „CloseCallback“).
- Die direkt oder indirekt durch einen Callback aufgerufenen Matlab-Funktionen können verschiedenen Dinge bewirken (Berechnungen starten, Importe ausführen, Diagramme erstellen, usw.), sie können insbesondere aber auch die GUI selber beeinflussen und verändern (z. B. über den `set`-Befehl).

4. Grafische Darstellung – 4.11 GUIs

Objektstruktur (Hierarchiestruktur):



4. Grafische Darstellung – 4.11 GUIs

Befehle für die Erstellung von Interaktiven Elementen:

- `uicontrol` → Erzeugung von Kontrollelementen
- `uimenu` → Erzeugung von Benutzermenüs
- `uicontextmenu` → Erzeugung von Kontextmenüs

Syntax: `uicontrol('Style', <style>, . . . );`

4. Grafische Darstellung – 4.11 GUIs

Mögliche Elemente (<style>):

checkbox	→	Anwahlfeld
edit	→	Textfeld editierbar
frame	→	Rahmen
listbox	→	Textauswahl
popupmenu	→	Aufklappmenü
pushbutton	→	Druckknopf
radiobutton	→	Auswahlfeld
slider	→	Rollbalken
text	→	Textfeld (nicht editierbar)
togglebutton	→	Umschalter

4. Grafische Darstellung – 4.11 GUIs

Beispiel:

```
figure
set(gcf,'menubar','none','units','normalized')

uimenu('label','Fenster schliessen','callback','close')

set(gca,'position',[.4 .1 .5 .8])

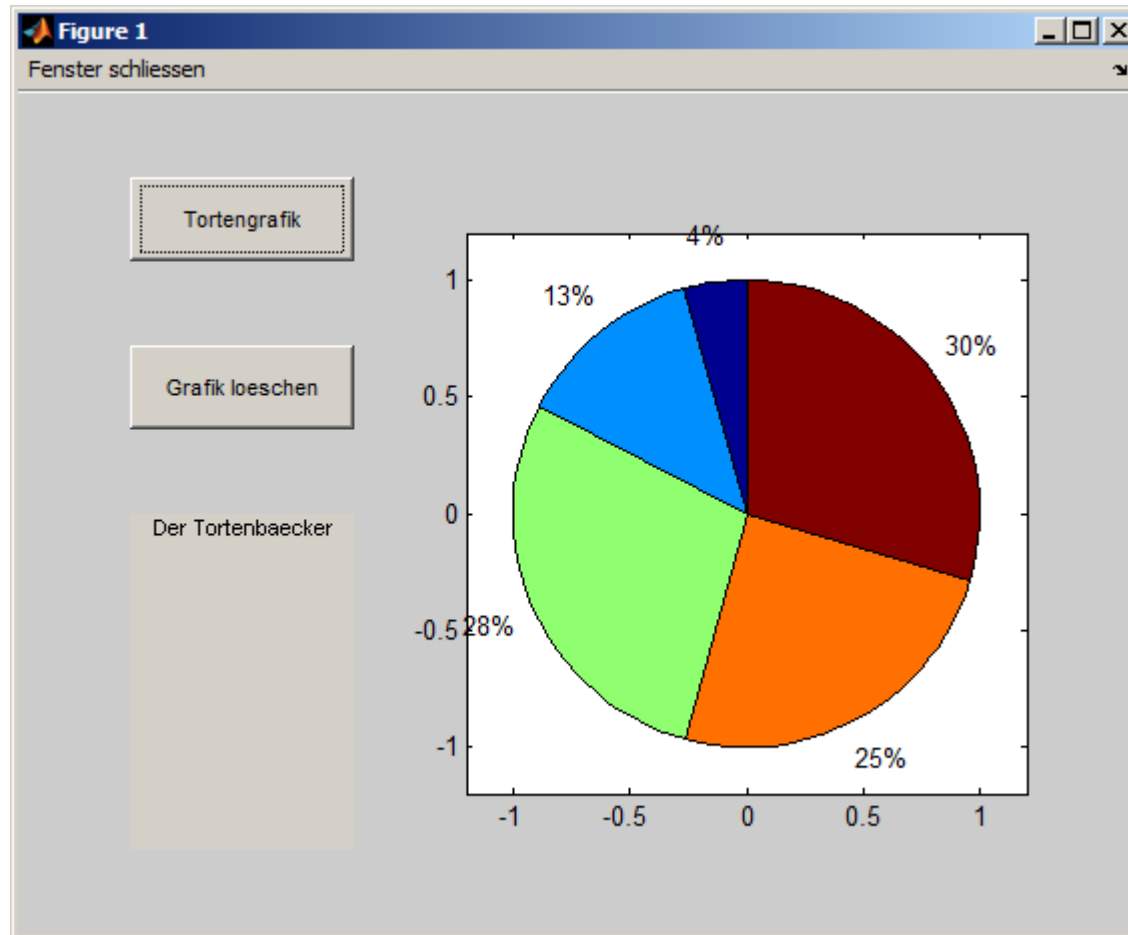
uicontrol('style','pushbutton','string','Tortengrafik',...
          'units','normalized','position',[.1 .8 .2 .1],...
          'callback','pie(rand(1,5)); axis on; box on;')

uicontrol('style','pushbutton','string','Grafik loeschen',...
          'units','normalized','position',[.1 .6 .2 .1],...
          'callback','cla')

uicontrol('style','text','string','Der Tortenbaecker',...
          'units','normalized','position',[.1 .1 .2 .4]);
```

4. Grafische Darstellung – 4.11 GUIs

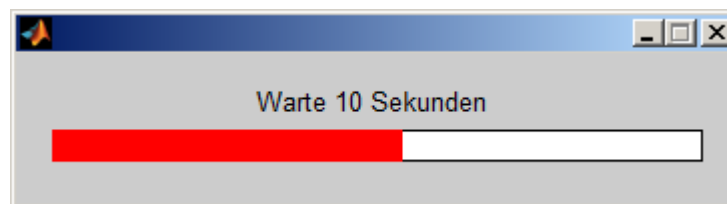
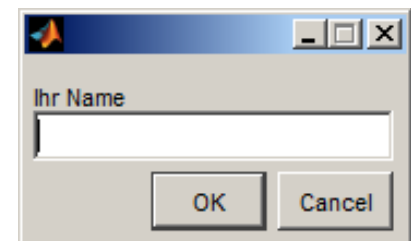
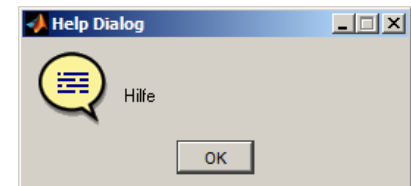
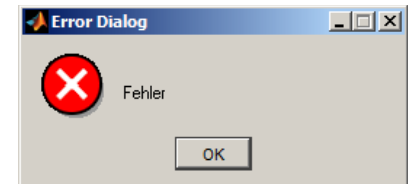
GUI:



4. Grafische Darstellung – 4.11 GUIs

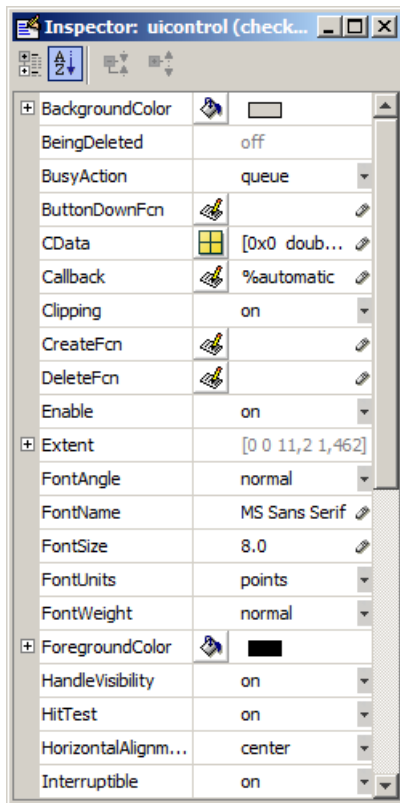
Standard Dialog-GUIs:

- `questdlg` → Dialogfenster für „Yes/No“-Abfrage
- `errordlg` → Dialogfenster für Fehlerausgabe
- `helpdlg` → Dialogfenster für Hilfetexte
- `inputdlg` → Dialogfenster für Eingaben
- `msgbox` → Dialogfenster für Mitteilungen
- `warndlg` → Dialogfenster für Warnungen
- `waitbar` → Fortschrittsbalken

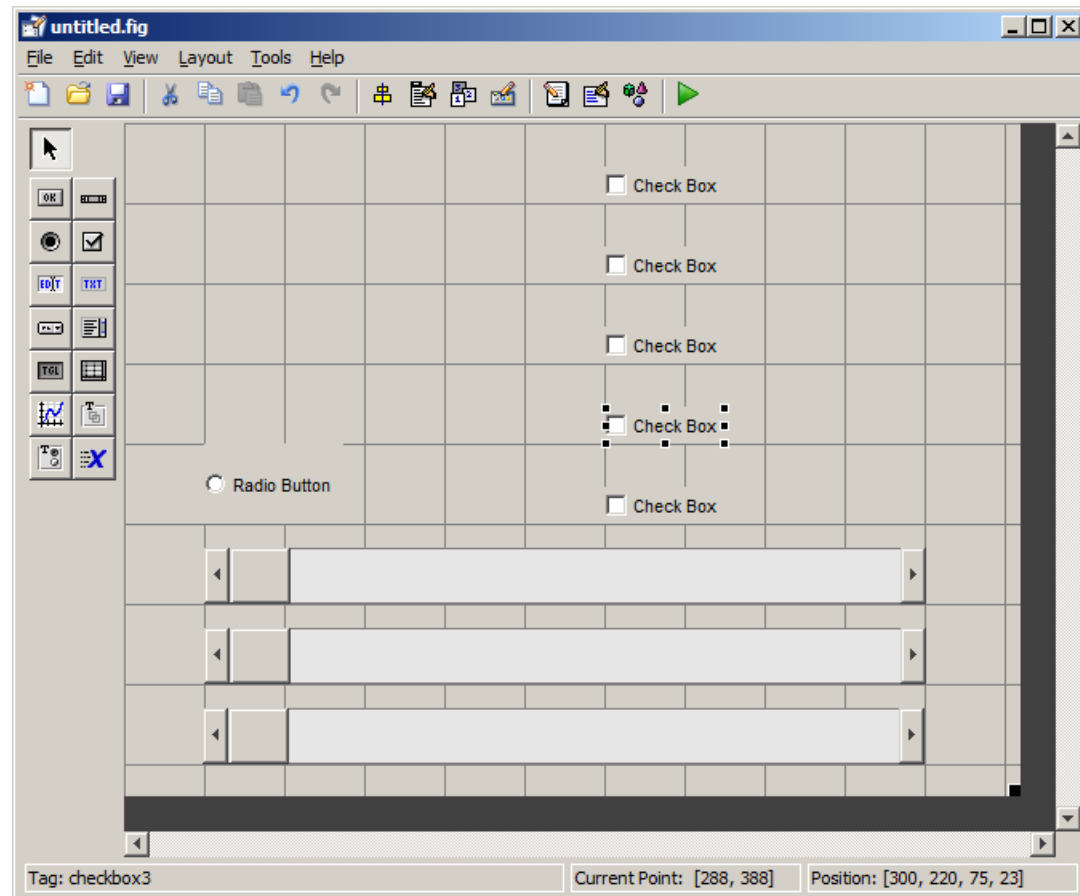


4. Grafische Darstellung – 4.11 GUIs

GUIs können außerdem interaktiv über den GUI-Editor erzeugt werden:
Der Editor wird mit der Anweisung `guide` aufgerufen.



Property
Inspector



Editor

4. Grafische Darstellung – 4.11 GUIs

Benutzung des Editors:

- Das Layout kann interaktiv gestalten und Kontrollelemente wie Diagrammflächen per drag-and-drop eingefügt werden.
- Der „Property Inspector“ ermöglicht das Einstellen der Objekteigenschaften die ansonsten über `get/set` zugänglich sind.
- Beim Speichern wird das Layout in einer `fig`-Datei abgelegt,
- Zusätzlich wird eine `m`-Datei erzeugt die bereit vorbereitete Callback-Funktionen enthält.

4. Grafische Darstellung – 4.11 GUIs

Weitere wichtige Befehle für GUIs:

<code>movegui</code>	→	Plazieren einer GUI auf dem Bildschirm
<code>openfig</code>	→	Öffnen einer fig-Datei
<code>waitfor</code>	→	Warten auf das Schließen einer GUI
<code>waitforbuttonpress</code>	→	Warten auf das betätigen eines Elements
<code>gcbf</code>	→	Handle des übergeordneten Grafikfensters
<code>gcbo</code>	→	Handle des Objekts des Callback ausgeführt w.
<code>uiwait</code>	→	Ausführung anhalten bis resume
<code>uiresume</code>	→	Ausführung fortsetzen

4. Grafische Darstellung – 4.11 GUIs

Hierarchien von Objekten:

Gruppieren von Objekten:

```

fhandle= figure;
set(gcf,'menubar','none','units','normalized');

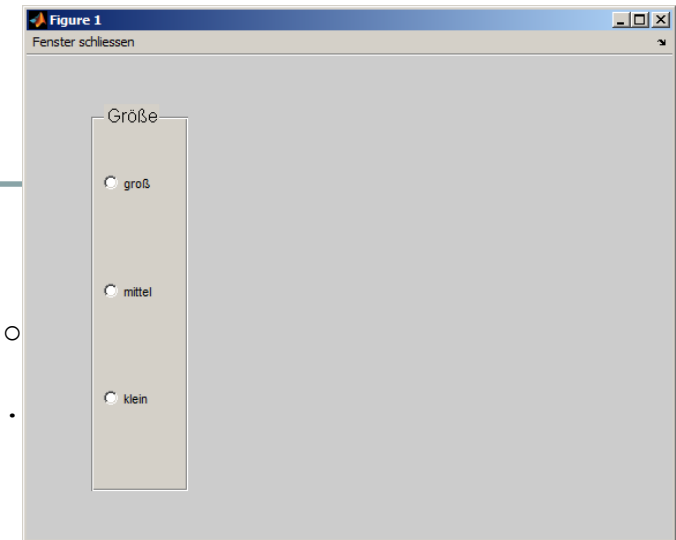
uimenu('label','Fenster schliessen','callback','clo

phandle = uipanel('Title','Größe','FontSize',12,...
    'Position',[.1 .1 .15 .8]);

grosshandle = uicontrol('style','Radiobutton',...
    'Parent',phandle,'string','groß',...
    'units','normalized','position',[.1 .8 .8 .1],...
    'callback','changesize(1)');

mittelhandle = uicontrol('style','Radiobutton',...
    'Parent',phandle,'string','mittel',...
    'units','normalized','position',[.1 .5 .8 .1],...
    'callback','changesize(2)');

kleinhandle = uicontrol('style','Radiobutton',...
    'Parent',phandle,'string','klein',...
    'units','normalized','position',[.1 .2 .8 .1],...
    'callback','changesize(3)');
  
```



4. Grafische Darstellung – 4.11 GUIs

Gruppierungen und Hierarchien:

- Grafikobjekte wie Kontrollelemente (`uicontrol`) und Grafiken (`axes`) oder Objekte in Grafiken (z.B. `plot`) können gruppiert werden und bilden dann unterhalb der Grafikfensters (`figure`) Hierarchien im Sinne von parent-child-Verhältnissen aus.
- Derartige Gruppierungen sind sinnvoll für die Handhabung der gruppierten Objekte (z. B. sichtbar/unsichtbar machen, verschieben, skalieren, drehen).
- Wichtige Befehle:

<code>allchild</code>	→	Ermitteln der untergeordneten Objekthandle
<code>hggroup</code>	→	Erzeugen einer Gruppe von Handlen
<code>hgtransform</code>	→	Transformieren einer Gruppe von Objekten